The Concise TypeScript Book

Simone Poggiali

The Concise TypeScript Book

The Concise TypeScript Book poskytuje ucelený a stručný přehled možností TypeScriptu. Nabízí srozumitelná vysvětlení všech aspektů nejnovější verze jazyka, od jeho výkonného typového systému až po pokročilé funkce.

Ať už jste začátečník, nebo zkušený vývojář, tato kniha je neocenitelným zdrojem pro prohloubení vašich znalostí a dovedností v TypeScriptu.

Tato kniha je zcela zdarma a má otevřený zdrojový kód.

Věřím, že kvalitní technické vzdělání by mělo být dostupné všem. Proto knihu ponechávám volně dostupnou a pravidelně ji aktualizuji o vylepšení a nové příklady.

Objevte **The Concise TypeScript Book Plus Edition**.



Pro čtenáře, kteří chtějí jít nad rámec open-source vydání, obsahuje **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** další a exkluzivní obsah zaměřený na praktické využití.

Plus Edition obsahuje:

- **Aktualizováno pro TypeScript 7** — přehled nejnovějších funkcí TypeScriptu 7 a vylepšení jazyka.
- **TypeScript s Reactem** — praktické rady pro typování komponent, props, hooků, událostí, potomků, refs a běžných vzorů v Reactu.

- **Recepty TypeScriptu pro skutečné projekty** — cílené příklady řešící praktické problémy, se kterými se vývojáři setkávají při tvorbě a údržbě aplikací v TypeScriptu.

Zakoupením Plus Edition také přímo podporujete další rozvoj a údržbu bezplatné open-source knihy.

Plus Edition je na Amazonu po celém světě k dispozici v angličtině a italštině. Prozkoumejte Plus Edition a kupte ji na Amazonu.

Podpořte projekt

Pokud vám bezplatná kniha pomohla opravit chybu, pochopit obtížný koncept nebo postoupit v kariéře, zvažte prosím podporu mé práce příspěvkem v libovolné výši, s doporučenou částkou **$5**, nebo mi kupte kávu.

Vaše podpora mi pomáhá udržovat obsah aktuální a rozšiřovat jej o nové příklady, srozumitelnější vysvětlení a další praktické rady.





Překlady

Tato kniha byla přeložena do několika jazyků, mezi něž patří:

Angličtina

Bulharština

Němčina

Francouzština

Indonéština

Italština

Japonština

Korejština

Polština

Portugalština (Brazílie)

Švédština

Turečtina

Vietnamština

Čínština

Španělština

Thajština

Ruština

Arabština Čeština

Soubory ke stažení a web

Můžete si také stáhnout verzi EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Online verze je k dispozici na adrese:

https://gibbok.github.io/typescript-book

Obsah

- The Concise TypeScript Book
  - Podpořte projekt
  - Překlady
  - Soubory ke stažení a web
  - Obsah
  - Úvod
  - O autorovi
  - Úvod do TypeScriptu
    - Co je TypeScript?
    - Proč TypeScript?
    - TypeScript a JavaScript
    - Generování kódu TypeScriptem
    - Moderní JavaScript už nyní (převod na starší verze)
  - Začínáme s TypeScriptem
    - Instalace
    - Konfigurace
    - Konfigurační soubor TypeScriptu
      - target
      - lib
      - strict
      - module
      - moduleResolution
      - esModuleInterop

        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Rady pro migraci na TypeScript
- Zkoumání typového systému
    - Jazyková služba TypeScriptu
    - Strukturální typování
    - Základní pravidla porovnávání v TypeScriptu
    - Typy jako množiny
    - Přiřazení typu: deklarace typů a přetypování
        - Deklarace typu
        - Přetypování
        - Ambientní deklarace
    - Kontrola vlastností a kontrola nadbytečných vlastností
    - Slabé typy
    - Přísná kontrola objektových literálů (čerstvost)
    - Odvozování typů
    - Pokročilejší odvozování typů
    - Rozšiřování typů
    - Const
        - Modifikátor const u typových parametrů
        - Přetypování const
    - Explicitní typová anotace
    - Zužování typů
        - Podmínky
        - Vyhození chyby nebo návrat
        - Diskriminované sjednocení
        - Uživatelsky definovaní typoví strážci

        - Zužování pomocí switch-true
- Primitivní typy
    - string
    - boolean
    - number
    - bigint
    - Symbol
    - null a undefined
    - Pole
    - any
- Typové anotace
- Volitelné vlastnosti
- Vlastnosti pouze pro čtení
- Indexové signatury
- Rozšíření typů
- Literálové typy
- Odvozování literálových typů
- strictNullChecks
- Výčtové typy
    - Číselné výčtové typy
    - Řetězcové výčtové typy
    - Konstantní výčtové typy
    - Zpětné mapování
    - Ambientní výčtové typy
    - Vypočítávané a konstantní členy
- Zužování typů
    - Typoví strážci typeof
    - Zužování podle pravdivostní hodnoty
    - Zužování podle rovnosti
    - Zužování pomocí operátoru in
    - Zužování pomocí instanceof
- Přiřazení

- Analýza toku řízení
- Typové predikáty
- Diskriminovaná sjednocení
- Datový typ never
- Kontrola úplnosti
- Objektové typy
- Typ n-tice (anonymní)
- Pojmenovaný typ n-tice (s popisky)
- N-tice s pevnou délkou
- Sjednocený typ
- Průnikové typy
- Indexování typů
- Typ z hodnoty
- Typ z návratové hodnoty funkce
- Typ z modulu
- Mapované typy
- Modifikátory mapovaných typů
- Podmíněné typy
- Distributivní podmíněné typy
- Odvozování typů pomocí infer v podmíněných typech
- Předdefinované podmíněné typy
- Šablonové sjednocené typy
- Typ any
- Typ unknown
- Typ void
- Typ never
- Rozhraní a typ
  - Obvyklá syntaxe
  - Základní typy
  - Objekty a rozhraní
  - Sjednocení a průniky typů
- Vestavěné primitivní typy

- Běžné vestavěné objekty JS
- Přetížení
- Slučování a rozšiřování
- Rozdíly mezi typem a rozhraním
- Třída
  - Obvyklá syntaxe třídy
  - Konstruktor
  - Soukromé a chráněné konstruktory
  - Modifikátory přístupu
  - Get a Set
  - Automatické přístupové metody ve třídách
  - this
  - Vlastnosti deklarované v parametrech
  - Abstraktní třídy
  - S generiky
  - Dekorátory
    - Dekorátory tříd
    - Dekorátor vlastnosti
    - Dekorátor metody
    - Dekorátory getterů a setterů
    - Metadata dekorátorů
  - Dědičnost
  - Statické členy
  - Inicializace vlastností
  - Přetěžování metod
- Generika
  - Generický typ
  - Generické třídy
  - Omezení generik
  - Kontextové zužování typů u generik
- Vymazané strukturální typy
- Jmenné prostory

- Symboly
- Direktivy s trojitým lomítkem
- Manipulace s typy
  - Vytváření typů z typů
  - Typy s indexovaným přístupem
  - Pomocné typy
    - Awaited<T>
    - Partial<T>
    - Required<T>
    - Readonly<T>
    - Record<K, T>
    - Pick<T, K>
    - Omit<T, K>
    - Exclude<T, U>
    - Extract<T, U>
    - NonNullable<T>
    - Parameters<T>
    - ConstructorParameters<T>
    - ReturnType<T>
    - InstanceType<T>
    - ThisParameterType<T>
    - OmitThisParameter<T>
    - ThisType<T>
    - Uppercase<T>
    - Lowercase<T>
    - Capitalize<T>
    - Uncapitalize<T>
    - NoInfer<T>
- Ostatní
  - Chyby a zpracování výjimek
  - Mixinové třídy
  - Asynchronní jazykové funkce

- Iterátory a generátory
- Referenční příručka TsDocs JSDoc
- @types
- JSX
- Moduly ES6
- Operátor umocňování ES7
- Příkaz for-await-of
- Metavlastnost new.target
- Výrazy dynamického importu
- “tsc –watch”
- Operátor potvrzení nenulové hodnoty
- Deklarace s výchozími hodnotami
- Volitelné řetězení
- Operátor slučování nulových hodnot
- Typy šablonových literálů
- Přetěžování funkcí
- Rekurzivní typy
- Rekurzivní podmíněné typy
- Podpora modulů ECMAScript v Node
- Aserční funkce
- Variadické typy n-tic
- Obalové typy
- Kovariance a kontravariance v TypeScriptu
  - Volitelné anotace variance pro typové parametry
- Indexové signatury se vzory šablonových řetězců
- Operátor satisfies
- Importy a exporty pouze typů
- Deklarace using a explicitní správa prostředků
  - Deklarace await using
- Atributy importu
- Kontrola syntaxe regulárních výrazů

    - import defer

Úvod

Vítejte v knize The Concise TypeScript Book! Tato příručka vás vybaví základními znalostmi a praktickými dovednostmi pro efektivní vývoj v TypeScriptu. Objevte klíčové koncepty a techniky pro psaní čistého a robustního kódu. Ať už jste začátečník, nebo zkušený vývojář, tato kniha slouží jako ucelený průvodce i praktická příručka pro využití možností TypeScriptu ve vašich projektech.

Tato kniha se věnuje TypeScriptu 7.0.

O autorovi

Simone Poggiali je zkušený Staff Engineer, který se s nadšením věnuje psaní kódu na profesionální úrovni již od 90. let. Během své mezinárodní kariéry přispěl k mnoha projektům pro širokou škálu klientů, od startupů až po velké organizace. Z jeho odborných znalostí a nasazení těžily významné společnosti jako HelloFresh, Siemens, O2, Leroy Merlin a Snowplow.

Simoneho Poggialiho můžete kontaktovat na následujících platformách:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- E-mail: gibbok.coding gmail.com

Úplný seznam přispěvatelů: https://github.com/gibbok/typescript-book/graphs/contributors

Úvod do TypeScriptu

Co je TypeScript?

TypeScript je silně typovaný programovací jazyk, který vychází z JavaScriptu. Původně jej v roce 2012 navrhl Anders Hejlsberg a v současnosti jej vyvíjí a udržuje Microsoft jako open-source projekt.

TypeScript se kompiluje do JavaScriptu a lze jej spustit v jakémkoli běhovém prostředí JavaScriptu (např. v prohlížeči nebo v Node.js na serveru).

Podporuje několik programovacích paradigmat, jako je funkcionální, generické, imperativní a objektově orientované programování, a jedná se o kompilovaný (transpilovaný) jazyk, který se před spuštěním převádí do JavaScriptu.

Proč TypeScript?

TypeScript je silně typovaný jazyk, který pomáhá předcházet běžným programátorským chybám a určitým druhům běhových chyb ještě před spuštěním programu.

Silně typovaný jazyk umožňuje vývojáři v definicích datových typů specifikovat různá omezení a chování programu, což usnadňuje ověřování správnosti softwaru a předcházení chybám. To je obzvláště cenné u rozsáhlých aplikací.

Některé výhody TypeScriptu:

- Statické typování, volitelně silné typování
- Odvozování typů

- Přístup k funkcím ES6 a ES7
- Kompatibilita napříč platformami a prohlížeči
- Podpora nástrojů s IntelliSense

TypeScript a JavaScript

TypeScript se píše do souborů `.ts` nebo `.tsx`, zatímco JavaScript do souborů `.js` nebo `.jsx`.

Soubory s příponou `.tsx` nebo `.jsx` mohou obsahovat rozšíření syntaxe JavaScriptu JSX, které se používá v Reactu pro vývoj uživatelského rozhraní.

TypeScript je z hlediska syntaxe typovanou nadmnožinou JavaScriptu (ECMAScript 2015). Veškerý kód JavaScriptu je platným kódem TypeScriptu, ale opačně to vždy neplatí.

Uvažujme například funkci v souboru JavaScriptu s příponou `.js`, jako je tato:

```
const sum = (a, b) => a + b;
```

Funkci lze převést a používat v TypeScriptu změnou přípony souboru na `.ts`. Pokud je však stejná funkce opatřena typovými anotacemi TypeScriptu, nelze ji bez kompilace spustit v žádném běhovém prostředí JavaScriptu. Následující kód TypeScriptu bez kompilace způsobí syntaktickou chybu:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript byl navržen tak, aby odhaloval potenciální běhové chyby při kompilaci tím, že vývojářům umožňuje vyjádřit záměr pomocí typových anotací. Díky odvozování typů navíc dokáže zachytit určité problémy i tehdy, když nejsou uvedeny

žádné explicitní typové anotace. Například následující ukázka kódu nespecifikuje žádné typy TypeScriptu:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

V tomto případě TypeScript odhalí chybu a oznámí:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Typový systém TypeScriptu je do značné míry ovlivněn chováním JavaScriptu za běhu. Například operátor sčítání (+), který v JavaScriptu může provádět buď spojování řetězců, nebo sčítání čísel, je v TypeScriptu modelován stejným způsobem:

```
const result = '1' + 1; // Výsledek je typu string
```

Tým stojící za TypeScriptem se záměrně rozhodl označovat neobvyklé použití JavaScriptu za chyby. Uvažujme například následující platný kód JavaScriptu:

```
const result = 1 + true; // V JavaScriptu je výsledek roven 2
```

TypeScript však ohlásí chybu:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Tato chyba nastává, protože TypeScript důsledně vynucuje kompatibilitu typů a v tomto případě identifikuje neplatnou operaci mezi číslem a logickou hodnotou.

Generování kódu TypeScriptem

Kompilátor TypeScriptu má dva hlavní úkoly: kontrolovat typové chyby a kompilovat do JavaScriptu. Tyto dva procesy jsou na sobě nezávislé. Typy neovlivňují vykonávání kódu v

běhovém prostředí JavaScriptu, protože jsou během kompilace zcela odstraněny. TypeScript může vygenerovat JavaScript i za přítomnosti typových chyb. Zde je příklad kódu TypeScriptu s typovou chybou:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument typu 'string' nelze přiřadit parametru typu 'number'.
```

Přesto může vzniknout spustitelný výstup v JavaScriptu:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Typy TypeScriptu nelze kontrolovat za běhu. Například:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
          // 'Dog' označuje pouze typ, ale zde se používá jako hodnota.
        // ...
    }
};
```

Protože jsou typy po kompilaci odstraněny, nelze tento kód spustit v JavaScriptu. K rozpoznávání typů za běhu musíme

použít jiný mechanismus. TypeScript nabízí několik možností, přičemž běžnou volbou je „označené sjednocení“. Například:

```
interface Dog {
    kind: 'dog'; // Diskriminované sjednocení
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Diskriminované sjednocení
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Vlastnost „kind“ je hodnota, kterou lze za běhu použít k rozlišení objektů v JavaScriptu.

Je také možné, aby hodnota měla za běhu jiný typ, než jaký byl uveden v deklaraci typu. Například pokud vývojář nesprávně pochopil typ z API a chybně jej anotoval.

TypeScript je nadmnožinou JavaScriptu, takže klíčové slovo „class“ lze použít jako typ i jako hodnotu za běhu.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

V JavaScriptu má „class“ vlastnost „prototype“ a operátor „instanceof“ lze použít k ověření, zda se vlastnost prototype konstruktoru nachází kdekoli v řetězci prototypů objektu.

TypeScript nemá vliv na výkon za běhu, protože všechny typy jsou odstraněny. Přináší však určitou režii při sestavování.

Moderní JavaScript už nyní (převod na starší verze)

TypeScript dokáže kompilovat kód do jakékoli vydané verze JavaScriptu od ECMAScriptu 3 (1999). To znamená, že umí převést kód využívající nejnovější funkce JavaScriptu do starších verzí; tento proces se označuje jako downleveling. Díky tomu lze používat moderní JavaScript a zároveň zachovat maximální kompatibilitu se staršími běhovými prostředími.

Je důležité poznamenat, že při transpilaci do starší verze JavaScriptu může TypeScript vygenerovat kód, který má oproti nativním implementacím určitou výkonnostní režii.

Zde jsou některé moderní funkce JavaScriptu, které lze v TypeScriptu používat:

- Moduly ECMAScript namísto callbacků „define“ ve stylu AMD nebo příkazů „require“ z CommonJS.
- Třídy namísto prototypů.
- Deklarace proměnných pomocí „let“ nebo „const“ namísto „var“.
- Cyklus „for-of“ nebo „.forEach“ namísto tradičního cyklu „for“.
- Šipkové funkce namísto funkčních výrazů.
- Destrukturovací přiřazení.
- Zkrácený zápis názvů vlastností a metod a vypočítávané názvy vlastností.
- Výchozí parametry funkcí.

Využitím těchto moderních funkcí JavaScriptu mohou vývojáři v TypeScriptu psát výstižnější a stručnější kód.

Začínáme s TypeScriptem

Instalace

Visual Studio Code poskytuje výbornou podporu jazyka TypeScript, ale neobsahuje jeho kompilátor. K instalaci kompilátoru TypeScriptu můžete použít správce balíčků, například npm nebo yarn:

```
npm install typescript --save-dev
```

nebo

```
yarn add typescript --dev
```

Nezapomeňte vytvořený soubor lockfile zahrnout do commitu, aby všichni členové týmu používali stejnou verzi TypeScriptu.

Kompilátor TypeScriptu můžete spustit následujícími příkazy:

```
npx tsc
```

nebo

```
yarn tsc
```

Doporučuje se instalovat TypeScript pro jednotlivé projekty namísto globální instalace, protože to zajišťuje předvídatelnější proces sestavování. Pro jednorázové použití však můžete použít následující příkaz:

```
npx tsc
```

nebo jej nainstalovat globálně:

```
npm install -g typescript
```

Pokud používáte Microsoft Visual Studio, můžete TypeScript pro své projekty MSBuild získat jako balíček v NuGet. V konzoli NuGet Package Manager spusťte následující příkaz:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Při instalaci TypeScriptu se nainstalují dva spustitelné soubory: „tsc“ jako kompilátor TypeScriptu a „tsserver“ jako samostatný server TypeScriptu. Samostatný server obsahuje kompilátor a jazykové služby, které mohou editory a IDE využívat k inteligentnímu doplňování kódu.

Kromě toho je k dispozici několik transpilátorů kompatibilních s TypeScriptem, například Babel (pomocí pluginu) nebo swc. Tyto transpilátory lze použít k převodu kódu TypeScriptu do jiných cílových jazyků nebo verzí.

TypeScript 7.0 byl přepsán do Go jako nativní implementace kompilátoru a jazykové služby. Využívá více vláken se sdílenou pamětí a další optimalizace ke zrychlení úplných sestavení a funkcí editoru, čímž zkracuje dobu čekání na zpětnou vazbu při vývoji.

Některé výkonnostní funkce TypeScriptu 7.0 lze nastavit. Kontrola typů může pomocí `--checkers` běžet v paralelních pracovních vláknech; více vláken může zrychlit velké projekty, ale spotřebuje více paměti. Přepracovaný režim `--watch` vylepšuje sledování souborů napříč platformami. TypeScript 7.0 zatím neobsahuje API kompilátoru (stav k červenci 2026), takže nástroje, které stále potřebují API TypeScriptu 6.0, mohou běžet souběžně s TypeScriptem 7.0 pomocí `@typescript/typescript6` nebo aliasů npm.

Konfigurace

TypeScript lze konfigurovat pomocí voleb příkazového řádku tsc nebo vyhrazeného konfiguračního souboru tsconfig.json umístěného v kořeni projektu.

K vygenerování souboru tsconfig.json předvyplněného doporučeným nastavením můžete použít následující příkaz:

```
tsc --init
```

Při místním spuštění příkazu `tsc` TypeScript zkompiluje kód s konfigurací uvedenou v nejbližším souboru tsconfig.json.

Zde je několik příkladů příkazů z příkazového řádku s výchozím nastavením:

```
tsc main.ts // Kompilace konkrétního souboru (main.ts) do JavaScriptu
tsc src/*.ts // Kompilace všech souborů .ts ve složce 'src' do JavaScriptu
tsc app.ts util.ts --outfile index.js // Kompilace dvou souborů TypeScriptu (app.ts a util.ts) do jednoho souboru JavaScriptu (index.js)
```

Konfigurační soubor TypeScriptu

Soubor tsconfig.json slouží ke konfiguraci kompilátoru TypeScriptu (tsc). Obvykle se přidává do kořene projektu společně se souborem `package.json`.

Poznámky:

- tsconfig.json podporuje komentáře, i když je ve formátu json.

- Doporučuje se používat tento konfigurační soubor namísto voleb příkazového řádku.

Na následujícím odkazu najdete úplnou dokumentaci a schéma:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Následuje seznam běžných a užitečných nastavení:

target

Vlastnost „target“ určuje, do jaké verze ECMAScriptu se má váš kód TypeScriptu vygenerovat či zkompilovat. Pro moderní prohlížeče je ES6 dobrou volbou. Poznámka: Podpora ES5 byla v TypeScriptu 6.0 označena za zastaralou a TypeScript 7.0 ji již nepodporuje.

lib

Vlastnost „lib“ určuje, které knihovní soubory mají být zahrnuty při kompilaci. TypeScript automaticky zahrnuje API pro funkce určené vlastností „target“, ale pro konkrétní potřeby lze některé knihovny vynechat nebo vybrat jednotlivé knihovny. Pokud například pracujete na serverovém projektu, můžete vyloučit knihovnu „DOM“, která je užitečná pouze v prostředí prohlížeče.

strict

Volba „strict“ zlepšuje typovou bezpečnost zapnutím přísnějších kontrol. Od TypeScriptu 6.0 je zapnutá ve výchozím nastavení;

jinak byste ji měli v tsconfig.json explicitně nastavit na true. Zapnutí „strict“ umožňuje TypeScriptu:

- Generovat kód s „use strict“ pro každý zdrojový soubor.
- Zohledňovat „null“ a „undefined“ při kontrole typů.
- Zakázat použití typu „any“, pokud nejsou přítomny typové anotace.
- Ohlásit chybu při použití výrazu „this“, který by jinak znamenal typ „any“.

module

Vlastnost „module“ nastavuje modulový systém podporovaný zkompilovaným programem. Za běhu se používá zavaděč modulů k nalezení a spuštění závislostí podle určeného modulového systému.

Nejběžnějšími zavaděči modulů používanými v JavaScriptu jsou CommonJS v Node.js pro serverové aplikace a RequireJS pro moduly AMD ve webových aplikacích běžících v prohlížeči. TypeScript dokáže generovat kód pro různé modulové systémy, včetně UMD, System, ESNext, ES2015/ES6 a ES2020. Modulový systém by měl být zvolen podle cílového prostředí a mechanismu načítání modulů dostupného v daném prostředí.

Poznámka: Podpora starších modulových systémů (AMD, UMD, SystemJS) byla v TypeScriptu 6.0 označena za zastaralou a TypeScript 7.0 ji již nepodporuje.

moduleResolution

Vlastnost „moduleResolution“ určuje strategii vyhledávání modulů. Pro moderní kód TypeScriptu používejte „nodenext“ nebo „bundler“. Strategie „classic“ se používá pouze pro staré verze TypeScriptu (před verzí 1.6).

esModuleInterop

Vlastnost „esModuleInterop“ umožňuje výchozí importy z modulů CommonJS, které neexportovaly pomocí vlastnosti „default“; tato vlastnost poskytuje vrstvu kompatibility ve vygenerovaném JavaScriptu. Po zapnutí této volby můžeme použít `import MyLibrary from "my-library"` namísto `import * as MyLibrary from "my-library"`.

„esModuleInterop“ se původně zapínalo volitelně, aby se předešlo nekompatibilním změnám, ale již dlouho je doporučeným výchozím nastavením. Jeho vypnutí může při používání CommonJS s ESM způsobit nenápadné problémy za běhu. Poznámka: Od TypeScriptu 6.0 je toto bezpečnější chování pro vzájemnou kompatibilitu vždy zapnuté.

V TypeScriptu 6.0 byly některé starší konfigurační volby a syntaktické formy označeny za zastaralé nebo procházely přechodným obdobím se starým chováním. V TypeScriptu 7.0 způsobují chyby, nebo nemají žádný účinek.

Zastaralé prvky, které se změnily na chyby a nemají žádný účinek, jsou:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`

- `baseUrl`
- `moduleResolution: classic`
- vypnutí `esModuleInterop` nebo `allowSyntheticDefaultImports`
- vypnutí `alwaysStrict`
- klíčové slovo `module` v deklaracích jmenných prostorů
- `asserts` u importů
- `/// <reference no-default-lib />` při použití `skipDefaultLibCheck`
- cesty k souborům v příkazovém řádku při existenci místního `tsconfig.json`, pokud není použito `--ignoreConfig`

jsx

Vlastnost „jsx“ se vztahuje pouze na soubory .tsx používané v ReactJS a řídí, jak se konstrukce JSX kompilují do JavaScriptu. Běžnou volbou je „preserve“, která při kompilaci vytvoří soubor .jsx a ponechá JSX beze změny, aby jej bylo možné předat dalším nástrojům, například Babelu, k dalším transformacím.

skipLibCheck

Vlastnost „skipLibCheck“ zabrání TypeScriptu v kontrole typů celých importovaných balíčků třetích stran. Tato vlastnost zkracuje dobu kompilace projektu. TypeScript bude i nadále kontrolovat váš kód proti definicím typů poskytovaným těmito balíčky.

files

Vlastnost „files“ předává kompilátoru seznam souborů, které musí být vždy zahrnuty do programu.

include

Vlastnost „include“ předává kompilátoru seznam souborů, které chceme zahrnout. Tato vlastnost umožňuje vzory podobné globům, například „*“ *pro libovolný podadresář*, „“ pro libovolný název souboru a „?“ pro volitelné znaky.

exclude

Vlastnost „exclude“ předává kompilátoru seznam souborů, které nemají být zahrnuty do kompilace. Může jít například o soubory „node_modules“ nebo testovací soubory. Poznámka: tsconfig.json umožňuje komentáře.

importHelpers

TypeScript používá pomocný kód při generování kódu pro některé pokročilé funkce JavaScriptu nebo při převodu na starší verze. Ve výchozím nastavení se tyto pomocné funkce duplikují v souborech, které je používají. Volba `importHelpers` je místo toho importuje z modulu `tslib`, čímž zefektivňuje výsledný JavaScript.

Rady pro migraci na TypeScript

U velkých projektů se doporučuje postupný přechod, při kterém budou zpočátku vedle sebe existovat kód TypeScriptu a kód JavaScriptu. Na TypeScript lze najednou převést pouze malé projekty.

Prvním krokem tohoto přechodu je začlenění TypeScriptu do procesu sestavování. To lze provést pomocí volby kompilátoru

„allowJs“, která umožňuje, aby soubory .ts a .tsx existovaly vedle stávajících souborů JavaScriptu. Protože TypeScript použije pro proměnnou typ „any“, pokud jej nedokáže odvodit ze souborů JavaScriptu, doporučuje se na začátku migrace ve volbách kompilátoru vypnout „noImplicitAny“.

Druhým krokem je zajistit, aby vaše testy JavaScriptu fungovaly společně se soubory TypeScriptu a abyste mohli testy spouštět při převodu jednotlivých modulů. Pokud používáte Jest, zvažte použití `ts-jest`, který umožňuje testovat projekty TypeScriptu pomocí Jestu.

Třetím krokem je přidání deklarací typů pro knihovny třetích stran do projektu. Tyto deklarace mohou být součástí knihoven nebo je lze najít na DefinitelyTyped. Můžete je vyhledat na https://www.typescriptlang.org/dt/search a nainstalovat pomocí:

```
npm install --save-dev @types/package-name
```

nebo

```
yarn add --dev @types/package-name
```

Čtvrtým krokem je migrace jednotlivých modulů zdola nahoru podle grafu závislostí, počínaje listy. Cílem je začít převodem modulů, které nezávisí na jiných modulech. K vizualizaci grafů závislostí můžete použít nástroj „madge“.

Vhodnými kandidáty pro tyto počáteční převody jsou pomocné funkce a kód související s externími API nebo specifikacemi. Definice typů TypeScriptu lze automaticky generovat z kontraktů Swagger, GraphQL nebo schémat JSON a zahrnout je do projektu.

Pokud nejsou k dispozici specifikace nebo oficiální schémata, můžete typy generovat ze surových dat, například z JSON vráceného serverem. Doporučuje se však generovat typy ze specifikací namísto dat, aby nebyly opomenuty okrajové případy.

Během migrace se vyhněte refaktoringu kódu a zaměřte se pouze na přidávání typů do modulů.

Pátým krokem je zapnutí „noImplicitAny“, které vynutí, aby všechny typy byly známé a definované, a tím zlepší práci s TypeScriptem ve vašem projektu.

Během migrace můžete použít direktivu `@ts-check`, která zapíná kontrolu typů TypeScriptu v souboru JavaScriptu. Tato direktiva poskytuje mírnější variantu kontroly typů a lze ji zpočátku využít k odhalování problémů v souborech JavaScriptu. Pokud je v souboru uvedeno `@ts-check`, TypeScript se pokusí odvodit definice pomocí komentářů ve stylu JSDoc. Použití anotací JSDoc však zvažujte pouze ve velmi rané fázi migrace.

Zvažte ponechání výchozí hodnoty `noEmitOnError` ve vašem tsconfig.json na false. To umožní vygenerovat zdrojový kód JavaScriptu i při ohlášených chybách.

Zkoumání typového systému

Jazyková služba TypeScriptu

Jazyková služba TypeScriptu, známá také jako tsserver, nabízí různé funkce, například hlášení chyb, diagnostiku, kompilaci při uložení, přejmenování, přechod na definici, seznamy

návrhů pro doplňování, nápovědu k signaturám a další. Používají ji především integrovaná vývojová prostředí (IDE) k poskytování podpory IntelliSense. Bezproblémově se integruje s Visual Studio Code a využívají ji nástroje jako Conquer of Completion (Coc).

Vývojáři mohou využít vyhrazené API a vytvářet vlastní pluginy jazykové služby pro zlepšení práce s kódem TypeScriptu. To může být obzvláště užitečné při implementaci speciálních funkcí lintu nebo při zajištění automatického doplňování pro vlastní šablonovací jazyk.

Příkladem vlastního pluginu používaného v praxi je „typescript-styled-plugin“, který poskytuje hlášení syntaktických chyb a podporu IntelliSense pro vlastnosti CSS ve styled components.

Další informace a úvodní návody najdete na oficiální wiki TypeScriptu na GitHubu: https://github.com/microsoft/TypeScript/wiki/

Strukturální typování

TypeScript je založen na strukturálním typovém systému. To znamená, že kompatibilita a ekvivalence typů se určují podle skutečné struktury nebo definice typu, nikoli podle jeho názvu či místa deklarace, jako je tomu v nominálních typových systémech, například v C# nebo C.

Strukturální typový systém TypeScriptu byl navržen podle toho, jak za běhu funguje dynamický systém duck typingu v JavaScriptu.

Následující příklad je platný kód TypeScriptu. Jak vidíte, „X“ a „Y“ mají stejný člen „a“, přestože jsou deklarovány pod různými názvy. Typy se určují podle svých struktur a v tomto případě jsou struktury stejné, takže jsou kompatibilní a kód je platný.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Platné
```

Základní pravidla porovnávání v TypeScriptu

Proces porovnávání v TypeScriptu je rekurzivní a provádí se na typech vnořených na libovolné úrovni.

Typ „X“ je kompatibilní s „Y“, pokud má „Y“ alespoň stejné členy jako „X“.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Platné, protože obsahuje alespoň stejné členy jako X
const r: X = y;
```

Parametry funkcí se porovnávají podle typů, nikoli podle názvů:

```
type X = (a: number) => void;
type Y = (a: number) => void;
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
```

```
y = x; // Platné
x = y; // Platné
```

Návratové typy funkcí musí být stejné:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Neplatné
x = y; // Neplatné
```

Návratový typ zdrojové funkce musí být podtypem návratového typu cílové funkce:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Platné
y = x; // Neplatné, chybí člen b
```

Vynechání parametrů funkce je povoleno, protože jde o běžnou praxi v JavaScriptu, například při použití „Array.prototype.map()“:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Proto jsou následující deklarace typů zcela platné:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Chybí parametr b
y = x; // Platné
```

Jakékoli další volitelné parametry zdrojového typu jsou platné:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
```

```
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Platné
x = y; //Platné
```

Jakékoli volitelné parametry cílového typu bez odpovídajících parametrů ve zdrojovém typu jsou platné a nepředstavují chybu:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Platné
x = y; // Platné
```

Se zbytkovým parametrem se zachází jako s nekonečnou řadou volitelných parametrů:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //platné
```

Funkce s přetížením jsou platné, pokud je signatura přetížení kompatibilní se signaturou implementace:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Platné
x('a', 1); // Platné

function y(a: string): void; // Neplatné, nekompatibilní se signaturou implementace
function y(a: string, b: number): void;
function y(a: string, b: number): void {
    console.log(a, b);
```

```
}
y('a');
y('a', 1);
```

Porovnání parametrů funkcí uspěje, pokud jsou zdrojové a cílové parametry přiřaditelné nadtypům nebo podtypům (bivariance).

```
// Nadtyp
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Podtyp
class Y extends X {}
// Podtyp
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance připouští nadtypy
console.log(getA(new X('x'))); // Platné
console.log(getA(new Y('Y'))); // Platné
console.log(getA(new Z('z'))); // Platné
```

Výčtové typy lze porovnávat s čísly a naopak a takové porovnání je platné, ale porovnání hodnot různých výčtových typů je neplatné.

```
enum X {
    A,
    B,
}
enum Y {
    A,
```

```
    B,
    C,
}
const xa: number = X.A; // Platné
const ya: Y = 0; // Platné
X.A === Y.A; // Neplatné
```

U instancí třídy se kontroluje kompatibilita jejich soukromých a chráněných členů:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Neplatné
```

Kontrola při porovnávání nezohledňuje odlišnou hierarchii dědičnosti, například:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
class Y extends X {
    public a: string;
    constructor(value: string) {
```

```
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Platné
x === z; // Platné, i když z pochází z jiné hierarchie dědičnosti
```

Generické typy se porovnávají podle struktury výsledného typu po uplatnění generického parametru; porovnává se pouze konečný výsledek jako negenerický typ.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Neplatné, protože se argument typu používá ve výsledné struktuře

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Platné, protože se argument typu nepoužívá ve výsledné struktuře
```

Pokud generické typy nemají určený typový argument, se všemi neurčenými argumenty se zachází jako s typem „any“:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Platné
```

Pamatujte:

```
let a: number = 1;
let b: number = 2;
a = b; // Platné, každý typ lze přiřadit sám sobě

let c: any;
c = 1; // Platné, všechny typy lze přiřadit typu any

let d: unknown;
d = 1; // Platné, všechny typy lze přiřadit typu unknown

let e: unknown;
let e1: unknown = e; // Platné, unknown lze přiřadit pouze sobě
a typu any
let e2: any = e; // Platné
let e3: number = e; // Neplatné

let f: never;
f = 1; // Neplatné, typu never nelze nic přiřadit

let g: void;
let g1: any;
g = 1; // Neplatné, void nelze přiřadit žádnému typu a žádný
typ nelze přiřadit typu void kromě any
g = g1; // Platné
```

Poznamenejme, že při zapnuté volbě „strictNullChecks“ se s „null“ a „undefined“ zachází podobně jako s „void“; jinak jsou podobné typu „never“.

Typy jako množiny

V TypeScriptu je typ množinou možných hodnot. Tato množina se také označuje jako obor hodnot typu. Každou hodnotu typu lze chápat jako prvek množiny. Typ stanovuje omezení, která musí každý prvek množiny splňovat, aby byl považován za jejího člena. Hlavním úkolem TypeScriptu je kontrolovat a ověřovat, zda je jedna množina podmnožinou druhé.

TypeScript podporuje různé druhy množin:

Pojem z teorie množin	**TypeScript**	**Poznámky**
Prázdná množina	never	„never“ obsahuje cokoli kromě sebe sama
Jednoprvková množina	undefined / null / literálový typ	
Konečná množina	boolean / sjednocení	
Nekonečná množina	string / number / object	
Univerzální množina	any / unknown	Každý prvek je členem „any“ a každá množina je jeho podmnožinou / „unknown“ je typově bezpečným protějškem „any“

Zde je několik příkladů:

TypeScript	**Pojem z teorie množin**	**Příklad**
never	∅ (prázdná množina)	const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’
Literálový typ	Jednoprvková množina	type X = ‘X’; type Y = 7;

Hodnota přiřaditelná T	Hodnota ∈ T (je prvkem)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’;
T1 přiřaditelný T2	T1 ⊆ T2 (je podmnožinou)	type XY = ‘X’ | ‘Y’; const x: XY = ‘X’; const j: XY = ‘J’; // Type “‘J”’ is not assignable to type ‘XY’.
T1 extends T2	T1 ⊆ T2 (je podmnožinou)	type X = ‘X’ extends string ? true : false;
T1 | T2	T1 ∪ T2 (sjednocení)	type XY = ‘X’ | ‘Y’; type JK = 1 | 2;
T1 & T2	T1 ∩ T2 (průnik)	type X = { a: string } type Y = { b: string } type XY = X & Y const x: XY = { a: ‘a’, b: ‘b’ }
unknown	Univerzální množina	const x: unknown = 1

Sjednocení (T1 | T2) vytváří širší množinu (obě množiny):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Platné
```

Průnik (T1 & T2) vytváří užší množinu (pouze společné prvky):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Neplatné
const j: XY = { a: 'a', b: 'b' }; // Platné
```

Klíčové slovo `extends` lze v tomto kontextu chápat jako „je podmnožinou“. Nastavuje omezení pro typ. Při použití s generickým typem omezuje `extends` generický typový parametr na konkrétnější typ.

Poznamenejme, že `extends` zde nijak nesouvisí s dědičností tříd ve smyslu OOP.

TypeScript pracuje se strukturálními typy a nemá přísnou nominální hierarchii. Ve skutečnosti se, stejně jako v příkladu níže, mohou dva typy překrývat, aniž by byl jeden podtypem druhého, protože TypeScript zohledňuje strukturu neboli tvar objektů.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
```

```
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Platné
```

Přiřazení typu: deklarace typů a přetypování

V TypeScriptu lze typ přiřadit různými způsoby:

Deklarace typu

V následujícím příkladu používáme x: X („: Type“) k deklaraci typu proměnné x.

```
type X = {
    a: string;
};

// Deklarace typu
const x: X = {
    a: 'a',
};
```

Pokud proměnná neodpovídá určenému formátu, TypeScript ohlásí chybu. Například:

```
type X = {
    a: string;
```

```
};

const x: X = {
    a: 'a',
     b: 'b', // Chyba: objektový literál může obsahovat pouze
známé vlastnosti
};
```

Přetypování

Přetypování lze přidat pomocí klíčového slova `as`. Tím sdělujeme kompilátoru, že vývojář má o typu více informací, a potlačujeme případné chyby.

Například:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

V uvedeném příkladu je objekt x pomocí klíčového slova as přetypován na typ X. To kompilátoru TypeScriptu sděluje, že objekt odpovídá určenému typu, přestože má navíc vlastnost b, která v definici typu není.

Přetypování je užitečné v situacích, kdy je nutné určit konkrétnější typ, zejména při práci s DOM. Například:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

Zde přetypování as HTMLInputElement sděluje TypeScriptu, že s výsledkem getElementById se má zacházet jako s HTMLInputElement. Přetypování lze použít také k přemapování klíčů, jak ukazuje příklad níže se šablonovými literály:

```
type J<Type> = {
    [
        Property in keyof Type as `prefix_${string & Property}`
    ]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

V tomto příkladu používá typ `J<Type>` mapovaný typ se šablonovým literálem k přemapování klíčů Type. Vytváří nové vlastnosti s „prefix_“ přidaným ke každému klíči a jejich odpovídajícími hodnotami jsou funkce vracející původní hodnoty vlastností.

Při použití přetypování TypeScript neprovádí kontrolu nadbytečných vlastností. Proto je obecně vhodnější použít deklaraci typu, pokud je struktura objektu známa předem.

Ambientní deklarace

Ambientní deklarace jsou soubory popisující typy kódu JavaScriptu a jejich názvy mají formát `.d.ts.`. Obvykle se importují a používají k anotaci existujících knihoven JavaScriptu nebo k přidání typů do stávajících souborů JS ve vašem projektu.

Typy mnoha běžných knihoven najdete na: https://github.com/DefinitelyTyped/DefinitelyTyped/

a lze je nainstalovat pomocí:

```
npm install --save-dev @types/library-name
```

Vámi definované ambientní deklarace můžete importovat pomocí odkazu se třemi lomítky:

```
/// <reference path="./library-types.d.ts" />
```

Ambientní deklarace můžete používat i v souborech JavaScriptu pomocí `// @ts-check`.

Klíčové slovo `declare` umožňuje definovat typy pro existující kód JavaScriptu bez jeho importu a slouží jako zástupný zápis pro typy z jiného souboru nebo z globálního prostoru.

Kontrola vlastností a kontrola nadbytečných vlastností

TypeScript je založen na strukturálním typovém systému, ale kontrola nadbytečných vlastností je jeho funkcí, která umožňuje ověřit, zda má objekt přesně ty vlastnosti, které jsou uvedeny v typu.

Kontrola nadbytečných vlastností se provádí například při přiřazování objektových literálů proměnným nebo při jejich předávání jako argumentů funkci s nadbytečnou vlastností.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
```

```
const x: X = y; // Platné díky strukturálnímu typování
const w: X = { a: 'a', b: 'b' }; // Neplatné kvůli kontrole
nadbytečných vlastností
```

Slabé typy

Typ je považován za slabý, pokud obsahuje pouze sadu vlastností, z nichž jsou všechny volitelné:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript považuje za chybu přiřazení čehokoli slabému typu, pokud neexistuje žádný překryv; například následující kód ohlásí chybu:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Neplatné
```

Ačkoli se to nedoporučuje, v případě potřeby lze tuto kontrolu obejít pomocí přetypování:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Platné
```

Nebo přidáním unknown do indexové signatury slabého typu:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Platné
```

Přísná kontrola objektových literálů (čerstvost)

Přísná kontrola objektových literálů, někdy označovaná jako „čerstvost“, je funkce TypeScriptu, která pomáhá zachytit nadbytečné vlastnosti nebo překlepy v jejich názvech, jež by při běžných kontrolách strukturálních typů zůstaly nepovšimnuty.

Při vytvoření objektového literálu jej kompilátor TypeScriptu považuje za „čerstvý“. Pokud je objektový literál přiřazen proměnné nebo předán jako parametr, TypeScript ohlásí chybu, jestliže objektový literál uvádí vlastnosti, které v cílovém typu neexistují.

„Čerstvost“ však zaniká, když je objektový literál rozšířen nebo je použito přetypování.

Zde je několik příkladů pro ilustraci:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Kontrola čerstvosti: neplatné přiřazení
var y: Y;
```

```
y = { a: 'a', bx: 'bx' }; // Kontrola čerstvosti: neplatné přiřazení

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Rozšíření: bez chyb, typy jsou strukturálně kompatibilní

fn({ a: 'a', bx: 'b' }); // Kontrola čerstvosti: neplatný argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Rozšíření: bez kontroly čerstvosti
```

Odvozování typů

Pokud není uvedena anotace, TypeScript dokáže odvodit typy při:

- Inicializaci proměnných.
- Inicializaci členů.
- Nastavování výchozích hodnot parametrů.
- Určování návratového typu funkce.

Například:

```
let x = 'x'; // Odvozený typ je string
```

Kompilátor TypeScriptu analyzuje hodnotu nebo výraz a určuje jejich typ na základě dostupných informací.

Pokročilejší odvozování typů

Pokud se při odvozování typů používá více výrazů, TypeScript hledá „nejlepší společné typy“. Například:

```
let x = [1, 'x', 1, null]; // Odvozený typ je: (string | number | null)[]
```

Pokud kompilátor nemůže najít nejlepší společné typy, vrátí sjednocení typů. Například:

```
let x = [new RegExp('x'), new Date()]; // Odvozený typ je: (RegExp | Date)[]
```

TypeScript k odvozování typů využívá „kontextové typování“ podle umístění proměnné. V následujícím příkladu kompilátor ví, že `e` je typu `MouseEvent`, díky typu události `click` definovanému v souboru lib.d.ts, který obsahuje ambientní deklarace různých běžných konstrukcí JavaScriptu a DOM:

```
window.addEventListener('click', function (e) {}); // Odvozený typ e je MouseEvent
```

Rozšiřování typů

Rozšiřování typů je proces, při kterém TypeScript přiřazuje typ proměnné inicializované bez typové anotace. Umožňuje přechod od užších typů k širším, ale nikoli naopak. V následujícím příkladu:

```
let x = 'x'; // TypeScript odvodí široký typ string
let y: 'y' | 'x' = 'y'; // Typ y je sjednocením literálových typů
y = x; // Neplatné: typ 'string' nelze přiřadit typu '"x" | "y"'.
```

TypeScript přiřazuje `string` proměnné `x` na základě jediné hodnoty uvedené při inicializaci (`x`); jde o příklad rozšiřování.

TypeScript nabízí způsoby, jak proces rozšiřování řídit, například pomocí „const“.

Const

Použití klíčového slova `const` při deklaraci proměnné vede v TypeScriptu k odvození užšího typu.

Například:

```
const x = 'x'; // TypeScript odvodí pro x užší typ 'x'
let y: 'y' | 'x' = 'y';
y = x; // Platné: typ x je odvozen jako 'x'
```

Použitím `const` k deklaraci proměnné x se její typ zúží na konkrétní literálovou hodnotu ‘x’. Protože je typ x zúžen, lze ji bez chyby přiřadit proměnné y. Typ lze odvodit proto, že proměnným `const` nelze znovu přiřazovat hodnoty, takže jejich typ lze zúžit na konkrétní literálový typ, v tomto případě na literálový typ ‘x’.

Modifikátor const u typových parametrů

Od verze TypeScriptu 5.0 lze u generického typového parametru uvést atribut `const`. To umožňuje odvodit nejpřesnější možný typ. Podívejme se na příklad bez použití `const`:

```
function identity<T>(value: T) {
    // Zde není const
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Odvozený typ
je: { a: string; b: string; }
```

Jak vidíte, u vlastností `a` a `b` je odvozen typ `string`.

Nyní se podívejme na rozdíl oproti verzi s `const`:

```
function identity<const T>(value: T) {
    // Použití modifikátoru const u parametrů typu
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Odvozený typ je: { a: "a"; b: "b"; }
```

Nyní vidíme, že vlastnosti `a` a `b` jsou odvozeny jako řetězcové literály, nikoli pouze jako typy `string`.

Přetypování const

Tato funkce umožňuje deklarovat proměnnou s přesnějším literálovým typem na základě její inicializační hodnoty a sdělit tak kompilátoru, že s hodnotou má zacházet jako s neměnným literálem. Zde je několik příkladů:

U jedné vlastnosti:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

U celého objektu:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

To může být obzvláště užitečné při definování typu n-tice:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // N-tice readonly [1, 2, 3]
```

Explicitní typová anotace

Můžeme být konkrétní a uvést typ. V následujícím příkladu je vlastnost `x` typu `number`:

```
const v = {
    x: 1, // Odvozený typ: number (rozšíření)
};
v.x = 3; // Platné
```

Typovou anotaci můžeme zpřesnit pomocí sjednocení literálových typů:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x je nyní sjednocením literálových typů: 1 | 2 | 3
};
v.x = 3; // Platné
v.x = 100; // Neplatné
```

Zužování typů

Zužování typů je proces v TypeScriptu, při kterém se obecný typ zužuje na konkrétnější. Dochází k tomu, když TypeScript analyzuje kód a určí, že určité podmínky nebo operace mohou zpřesnit informace o typu.

K zužování typů může docházet různými způsoby, mezi něž patří:

Podmínky

Pomocí podmíněných příkazů, jako je `if` nebo `switch`, může TypeScript zúžit typ na základě výsledku podmínky. Například:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // Typ je number, zúžený podmínkou
}
```

Vyhození chyby nebo návrat

Vyhození chyby nebo předčasný návrat z větve lze využít k tomu, aby TypeScript snáze zúžil typ. Například:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Mezi další způsoby zužování typů v TypeScriptu patří:

- Operátor `instanceof`: Slouží ke kontrole, zda je objekt instancí konkrétní třídy.
- Operátor `in`: Slouží ke kontrole, zda v objektu existuje vlastnost.
- Operátor `typeof`: Slouží ke kontrole typu hodnoty za běhu.
- Vestavěné funkce jako `Array.isArray()`: Slouží ke kontrole, zda je hodnota pole.

Diskriminované sjednocení

„Diskriminované sjednocení“ je vzor v TypeScriptu, při kterém se k objektům přidává explicitní „značka“ pro rozlišení jednotlivých typů ve sjednocení. Tento vzor se také označuje jako „označené sjednocení“. V následujícím příkladu je „značka“ reprezentována vlastností „type“:

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // typ je A
        case 'type_b':
            return input.value + 'extra'; // typ je B
    }
};
```

Uživatelsky definovaní typoví strážci

V případech, kdy TypeScript nedokáže určit typ, lze napsat pomocnou funkci známou jako „uživatelsky definovaný typový strážce“. V následujícím příkladu použijeme typový predikát ke zúžení typu po provedení určitého filtrování:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // Typ je (string | null)[], TypeScript nedokázal správně odvodit typ

const isValid = (item: string | null): item is string => item !== null; // Vlastní strážce typu

const r2 = data.filter(isValid); // Typ je nyní správně string[], pomocí typového predikátu se podařilo typ zúžit
```

Zužování pomocí switch-true

TypeScript 5.3 přidává zužování pomocí switch-true, které umožňuje nahradit nepřehledné řetězce if/else konstrukcí switch (true) s logickými podmínkami. Zlepšuje čitelnost a přitom stále zužuje typy. Podobá se porovnávání vzorů, ale je jednodušší.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
    }
}
```

Primitivní typy

TypeScript podporuje 7 primitivních typů. Primitivní datový typ označuje typ, který není objektem a nemá žádné přidružené metody. V TypeScriptu jsou všechny primitivní typy neměnné, což znamená, že jejich hodnoty nelze po přiřazení změnit.

string

Primitivní typ `string` uchovává textová data a jeho hodnota je vždy uzavřena do dvojitých nebo jednoduchých uvozovek.

```
const x: string = 'x';
const y: string = 'y';
```

Řetězce mohou zabírat více řádků, pokud jsou ohraničeny znakem zpětného apostrofu (`):

```
let sentence: string = `xxx,
   yyy`;
```

boolean

Datový typ `boolean` v TypeScriptu uchovává binární hodnotu, buď `true`, nebo `false`.

```
const isReady: boolean = true;
```

number

Datový typ `number` v TypeScriptu je reprezentován 64bitovou hodnotou s plovoucí desetinnou čárkou. Typ `number` může představovat celá čísla i zlomky. TypeScript podporuje také šestnáctkový, dvojkový a osmičkový zápis, například:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Šestnáctkové číslo začíná na 0x
const binary: number = 0b1010; // Dvojkové číslo začíná na 0b
const octal: number = 0o633; // Osmičkové číslo začíná na 0o
```

bigint

Typ `bigint` představuje celočíselné hodnoty, které mohou být větší než nejvyšší bezpečné celé číslo podporované typem `number`, tedy 2^53 - 1.

Hodnotu `bigint` lze vytvořit voláním vestavěné funkce `BigInt()` nebo přidáním `n` na konec libovolného celočíselného literálu:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Poznámky:

- Hodnoty `bigint` nelze kombinovat s hodnotami `number` ani používat s vestavěným objektem `Math`; je nutné je převést na stejný typ.
- Hodnoty `bigint` jsou dostupné pouze tehdy, pokud je cílová konfigurace ES2020 nebo vyšší.

Symbol

Symboly jsou jedinečné identifikátory, které lze použít jako klíče vlastností v objektech, aby se předešlo kolizím názvů.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

null a undefined

Typy `null` a `undefined` představují chybějící hodnotu nebo nepřítomnost jakékoli hodnoty.

Typ `undefined` znamená, že hodnota není přiřazena nebo inicializována, případně označuje neúmyslnou nepřítomnost

hodnoty.

Typ `null` znamená, že víme, že daná položka nemá hodnotu, takže hodnota není dostupná, a označuje úmyslnou nepřítomnost hodnoty.

Pole

Typ `array` je datový typ, který může uchovávat více hodnot stejného i různého typu. Lze jej definovat následující syntaxí:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
const j: Array<string | number> = ['a', 1, 'b', 2]; // Sjednocení
```

TypeScript podporuje pole pouze pro čtení pomocí následující syntaxe:

```
const x: readonly string[] = ['a', 'b']; // Modifikátor readonly
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Neplatné
```

TypeScript podporuje n-tice a n-tice pouze pro čtení:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

any

Datový typ `any` představuje doslova „jakoukoli“ hodnotu a je výchozí, pokud TypeScript nedokáže typ odvodit nebo pokud typ není uveden.

Při použití `any` kompilátor TypeScriptu vynechává kontrolu typů, takže použití `any` neposkytuje typovou bezpečnost. Obecně nepoužívejte `any` k umlčení kompilátoru při výskytu chyby; místo toho se zaměřte na její opravu, protože použití `any` umožňuje porušit kontrakty a přijít o výhody automatického doplňování v TypeScriptu.

Typ `any` může být užitečný při postupné migraci z JavaScriptu na TypeScript, protože dokáže umlčet kompilátor.

U nových projektů používejte konfiguraci TypeScriptu `noImplicitAny`, která umožňuje TypeScriptu hlásit chyby tam, kde je `any` použit nebo odvozen.

Typ `any` je obvykle zdrojem chyb, které mohou skrývat skutečné problémy s vašimi typy. Vyhýbejte se jeho použití, jak jen to je možné.

Typové anotace

U proměnných deklarovaných pomocí `var`, `let` a `const` lze volitelně přidat typ:

```
const x: number = 1;
```

TypeScript dobře odvozuje typy, zejména ty jednoduché, takže tyto deklarace nejsou ve většině případů nutné.

U funkcí lze k parametrům přidat typové anotace:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Následuje příklad s anonymní funkcí (nazývanou také lambda funkce):

```
const sum = (a: number, b: number) => a + b;
```

Tyto anotace lze vynechat, pokud má parametr výchozí hodnotu:

```
const sum = (a = 10, b: number) => a + b;
```

K funkcím lze přidat anotace návratového typu:

```
const sum = (a = 10, b: number): number => a + b;
```

To je užitečné zejména u složitějších funkcí, protože zápis návratového typu před implementací vám může pomoci funkci promyslet.

Obecně zvažte anotování typových signatur, ale ne lokálních proměnných v těle funkce, a k objektovým literálům vždy přidávejte typy.

Volitelné vlastnosti

Objekt může určit volitelné vlastnosti přidáním otazníku `?` na konec názvu vlastnosti:

```
type X = {
    a: number;
    b?: number; // Volitelné
};
```

Pokud je vlastnost volitelná, lze zadat výchozí hodnotu:

```
type X = {
    a: number;
```

```
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

Vlastnosti pouze pro čtení

Zápisu do vlastnosti lze zabránit pomocí modifikátoru `readonly`, který zajišťuje, že vlastnost nelze přepsat, ale neposkytuje žádnou záruku úplné neměnnosti:

```
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

Indexové signatury

V TypeScriptu můžeme v indexových signaturách používat `string`, `number` a `symbol`:

```
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
```

```
console.log(k[1]);
console.log(k['1']); // Stejný výsledek jako k[1]
```

Pamatujte, že JavaScript automaticky převádí index typu `number` na index typu `string`, takže `k[1]` i `k["1"]` vracejí stejnou hodnotu.

Rozšíření typů

Je možné rozšířit `interface` (zkopírovat členy z jiného typu):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Je také možné dědit z více typů:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Klíčové slovo `extends` funguje pouze u rozhraní a tříd; u typů použijte průnik:

```
type A = {
    a: number;
};
type B = {
    b: number;
```

```
};
type C = A & B;
```

Typ lze rozšířit pomocí rozhraní, ale ne naopak:

```
type A = {
    a: string;
};
interface B extends A {
    b: string;
}
```

Literálové typy

Literálový typ je jednoprvková množina v rámci obecnějšího typu; definuje zcela konkrétní hodnotu, která je primitivem JavaScriptu.

Literálové typy v TypeScriptu jsou čísla, řetězce a booleovské hodnoty.

Příklad literálů:

```
const a = 'a'; // Řetězcový literálový typ
const b = 1; // Číselný literálový typ
const c = true; // Booleovský literálový typ
```

Řetězcové, číselné a booleovské literálové typy se používají ve sjednoceních, typových strážcích a typových aliasech. V následujícím příkladu vidíte typový alias sjednocení. `O` se skládá pouze z uvedených hodnot, žádný jiný řetězec není platný:

```
type O = 'a' | 'b' | 'c';
```

Odvozování literálových typů

Odvozování literálových typů je funkce TypeScriptu, která umožňuje odvodit typ proměnné nebo parametru na základě jeho hodnoty.

V následujícím příkladu vidíme, že TypeScript považuje `x` za literálový typ, protože hodnotu již později nelze změnit, zatímco u `y` odvodí typ string, protože ji lze později kdykoli upravit.

```
const x = 'x'; // Literálový typ 'x', protože tuto hodnotu nelze změnit
let y = 'y'; // Typ string, protože tuto hodnotu lze změnit
```

V následujícím příkladu vidíme, že typ `o.x` byl odvozen jako `string` (a nikoli jako literál `a`), protože TypeScript předpokládá, že hodnotu lze později kdykoli změnit.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // Toto je širší typ string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument typu 'string' nelze přiřadit parametru typu 'X'
```

Jak vidíte, kód při předání `o.x` do `fn` vyvolá chybu, protože X je užší typ.

Tento problém můžeme vyřešit přetypováním pomocí `const` nebo typu `X`:

```
let o = {
    x: 'a' as const,
```

```
};
```

nebo:

```
let o = {
    x: 'a' as X,
};
```

strictNullChecks

`strictNullChecks` je volba kompilátoru TypeScriptu, která vynucuje striktní kontrolu hodnot null. Pokud je tato volba zapnuta, lze proměnným a parametrům přiřadit `null` nebo `undefined` pouze tehdy, pokud byly explicitně deklarovány jako tento typ pomocí sjednoceného typu `null | undefined`. Pokud proměnná nebo parametr nejsou explicitně deklarovány jako umožňující hodnotu null, TypeScript vygeneruje chybu, aby zabránil potenciálním chybám za běhu.

Výčtové typy

V TypeScriptu je `enum` množina pojmenovaných konstantních hodnot.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Výčtové typy lze definovat různými způsoby:

Číselné výčtové typy

Číselný výčtový typ v TypeScriptu je výčtový typ, v němž je každé konstantě přiřazena číselná hodnota, přičemž číslování ve výchozím nastavení začíná od 0.

```
enum Size {
    Small, // hodnota začíná od 0
    Medium,
    Large,
}
```

Vlastní hodnoty lze zadat jejich explicitním přiřazením:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

Řetězcové výčtové typy

Řetězcový výčtový typ v TypeScriptu je výčtový typ, v němž je každé konstantě přiřazena řetězcová hodnota.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Poznámka: TypeScript umožňuje použití heterogenních výčtových typů, v nichž mohou současně existovat řetězcové i číselné členy.

Konstantní výčtové typy

Konstantní výčtový typ v TypeScriptu je speciální druh výčtového typu, v němž jsou všechny hodnoty známé v době překladu a jsou přímo vloženy všude, kde je výčtový typ použit, což vede k efektivnějšímu kódu.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Přeloží se do:

```
console.log('EN' /* Language.English */);
```

Poznámky: Konstantní výčtové typy mají hodnoty vložené přímo do kódu a samotný výčtový typ je odstraněn, což může být efektivnější v samostatných knihovnách, ale obecně to není žádoucí. Konstantní výčtové typy navíc nemohou mít vypočítávané členy.

Zpětné mapování

Zpětné mapování u výčtových typů v TypeScriptu označuje možnost získat název členu výčtového typu z jeho hodnoty. Ve výchozím nastavení mají členy výčtového typu dopředné mapování z názvu na hodnotu, ale zpětné mapování lze vytvořit explicitním nastavením hodnot pro každého člena. Zpětné mapování je užitečné, když potřebujete vyhledat člena výčtového typu podle jeho hodnoty nebo procházet všechny členy výčtového typu. Pamatujte, že zpětné mapování se generuje pouze pro číselné členy výčtového typu, zatímco pro řetězcové členy se negeneruje vůbec.

Následující výčtový typ:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
```

se přeloží do:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Mapování hodnot na klíče tedy funguje pro číselné členy výčtového typu, ale ne pro řetězcové členy:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Prvek  má  implicitně  typ
'any', protože indexový výraz není typu 'number'.
```

Ambientní výčtové typy

Ambientní výčtový typ v TypeScriptu je druh výčtového typu definovaný v deklaračním souboru (*.d.ts) bez přidružené implementace. Umožňuje definovat množinu pojmenovaných konstant, které lze typově bezpečně používat napříč různými soubory, aniž by bylo nutné do každého souboru importovat detaily implementace.

Vypočítávané a konstantní členy

V TypeScriptu je vypočítávaný člen členem výčtového typu, jehož hodnota se počítá za běhu, zatímco konstantní člen je člen, jehož hodnota je nastavena v době překladu a za běhu ji nelze změnit. Vypočítávané členy jsou povoleny v běžných výčtových typech, zatímco konstantní členy jsou povoleny v běžných i konstantních výčtových typech.

```
// Konstantní členy
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6, vypočteno při kompilaci

// Vypočítávané členy
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // náhodné číslo generované za běhu
```

Výčtové typy jsou vyjádřeny sjednoceními tvořenými typy jejich členů. Hodnoty jednotlivých členů lze určit pomocí konstantních nebo nekonstantních výrazů, přičemž členům s konstantními hodnotami jsou přiřazeny literálové typy. Pro ilustraci zvažte deklaraci typu E a jeho podtypů E.A, E.B a E.C. V tomto případě E představuje sjednocení E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Číselný literál
    B = 'bar', // Řetězcový literál
    C = identity(42), // Neprůhledná vypočítávaná hodnota
}

console.log(E.C); //42
```

Zužování typů

Zužování typů v TypeScriptu je proces zpřesňování typu proměnné uvnitř podmíněného bloku. To je užitečné při práci se sjednocenými typy, kdy proměnná může mít více než jeden typ.

TypeScript rozpoznává několik způsobů zužování typů:

Typoví strážci typeof

Typový strážce typeof je konkrétní typový strážce v TypeScriptu, který kontroluje typ proměnné na základě jejího vestavěného typu v JavaScriptu.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x je typu number
```

```
    }
    return -1;
};
```

Zužování podle pravdivostní hodnoty

Zužování podle pravdivostní hodnoty v TypeScriptu funguje tak, že kontroluje, zda se proměnná vyhodnocuje jako pravdivá (truthy), nebo nepravdivá (falsy), a podle toho zužuje její typ.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

Zužování podle rovnosti

Zužování podle rovnosti v TypeScriptu funguje tak, že kontroluje, zda se proměnná rovná konkrétní hodnotě, či nikoli, a podle toho zužuje její typ.

Používá se ve spojení s příkazy `switch` a operátory rovnosti, jako jsou `===`, `!==`, `==` a `!=`, k zužování typů.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

Zužování pomocí operátoru in

Zužování pomocí operátoru `in` v TypeScriptu je způsob, jak zúžit typ proměnné na základě toho, zda v jejím typu existuje určitá vlastnost.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

Zužování pomocí instanceof

Zužování pomocí operátoru `instanceof` v TypeScriptu je způsob, jak zúžit typ proměnné na základě její konstruktorové funkce kontrolou, zda je objekt instancí určité třídy nebo rozhraní.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
```

```
    ) {}
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

Přiřazení

Zužování typů pomocí přiřazení v TypeScriptu je způsob, jak zúžit typ proměnné na základě hodnoty, která je jí přiřazena. Při přiřazení hodnoty proměnné TypeScript odvodí její typ a zúží jej tak, aby odpovídal přiřazené hodnotě.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

Analýza toku řízení

Analýza toku řízení v TypeScriptu je způsob statické analýzy toku kódu za účelem odvození typů proměnných, který

umožňuje kompilátoru podle potřeby zužovat typy těchto proměnných na základě výsledků analýzy.

Před TypeScriptem 4.4 se analýza toku kódu vztahovala pouze na kód uvnitř příkazu if, ale od TypeScriptu 4.4 ji lze použít také na podmíněné výrazy a přístupy k diskriminačním vlastnostem, na které se nepřímo odkazuje prostřednictvím proměnných const.

Například:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Několik příkladů, kdy k zužování nedochází:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
         x.length; // Chyba, bez zúžení, protože isString není
const
```

```
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Chyba, bez zúžení, protože obj se přiřazuje
v těle funkce
    }
};
```

Poznámky: V podmíněných výrazech se analyzuje až pět úrovní nepřímých odkazů.

Typové predikáty

Typové predikáty v TypeScriptu jsou funkce, které vracejí booleovskou hodnotu a slouží k zúžení typu proměnné na konkrétnější typ.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 automaticky odvozuje typové predikáty (například `x is T`) ve funkcích, jako je `.filter`, takže rozpozná,

kdy jsou odstraněny hodnoty jako undefined, což poskytuje přesnější typy a méně chyb; to funguje pro jednoznačné kontroly (např. `x !== undefined`), ale ne pro nejednoznačné, jako je `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

Diskriminovaná sjednocení

Diskriminovaná sjednocení v TypeScriptu jsou druhem sjednoceného typu, který používá společnou vlastnost, označovanou jako diskriminant, k zúžení množiny možných typů ve sjednocení.

```
type Square = {
    kind: 'square'; // Diskriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Diskriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};

const square: Square = { kind: 'square', size: 5 };
```

```
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

Datový typ never

Když je proměnná zúžena na typ, který nemůže obsahovat žádné hodnoty, kompilátor TypeScriptu odvodí, že proměnná musí být typu `never`. Je to proto, že typ never představuje hodnotu, která nikdy nemůže vzniknout.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
        // val má zde typ never, protože nemůže být ničím jiným než řetězcem nebo číslem
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

Kontrola úplnosti

Kontrola úplnosti je funkce TypeScriptu, která zajišťuje, že jsou v příkazu `switch` nebo `if` ošetřeny všechny možné případy diskriminovaného sjednocení.

```
type Direction = 'up' | 'down';

const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
```

```
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // Tento řádek se
nikdy neprovede
    }
};
```

Typ `never` slouží k zajištění úplnosti výchozí větve a toho, že TypeScript vyvolá chybu, pokud je do typu Direction přidána nová hodnota, která není ošetřena v příkazu switch.

Objektové typy

Objektové typy v TypeScriptu popisují strukturu objektu. Určují názvy a typy jeho vlastností i to, zda jsou tyto vlastnosti povinné, nebo volitelné.

V TypeScriptu můžete objektové typy definovat dvěma hlavními způsoby:

Rozhraní definuje strukturu objektu určením názvů, typů a volitelnosti jeho vlastností.

```
interface User {
    name: string;
    age: number;
    email?: string;
}
```

Typový alias podobně jako rozhraní definuje strukturu objektu. Může však také vytvořit nový vlastní typ založený na

existujícím typu nebo kombinaci existujících typů. To zahrnuje definování sjednocených typů, průnikových typů a dalších složitých typů.

```
type Point = {
    x: number;
    y: number;
};
```

Typ lze také definovat anonymně:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

Typ n-tice (anonymní)

Typ n-tice je typ, který představuje pole s pevným počtem prvků a jejich odpovídajícími typy. Typ n-tice vynucuje konkrétní počet prvků a jejich příslušné typy v pevném pořadí. Typy n-tic jsou užitečné, když chcete reprezentovat kolekci hodnot s konkrétními typy, kde má pozice každého prvku v poli určitý význam.

```
type Point = [number, number];
```

Pojmenovaný typ n-tice (s popisky)

Typy n-tic mohou obsahovat volitelné popisky nebo názvy pro každý prvek. Tyto popisky slouží ke zlepšení čitelnosti a usnadňují práci nástrojům a neovlivňují operace, které s nimi můžete provádět.

```
type T = string;
type Tuple1 = [T, T];
```

```
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Pojmenovaná n-tice spolu s anonymní n-ticí
```

N-tice s pevnou délkou

N-tice s pevnou délkou je konkrétní druh n-tice, který vynucuje pevný počet prvků určitých typů a po definování neumožňuje žádné změny délky n-tice.

N-tice s pevnou délkou jsou užitečné, když potřebujete reprezentovat kolekci hodnot s konkrétním počtem prvků a konkrétními typy a chcete zajistit, aby délku a typy n-tice nebylo možné neúmyslně změnit.

```
const x = [10, 'hello'] as const;
x.push(2); // Chyba
```

Sjednocený typ

Sjednocený typ je typ, který představuje hodnotu, jež může být jedním z několika typů. Sjednocené typy se zapisují pomocí symbolu `|` mezi jednotlivými možnými typy.

```
let x: string | number;
x = 'hello'; // Platné
x = 123; // Platné
```

Průnikové typy

Průnikový typ je typ, který představuje hodnotu se všemi vlastnostmi dvou nebo více typů. Průnikové typy se zapisují pomocí symbolu `&` mezi jednotlivými typy.

```ts
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Průnik

const j: J = {
    a: 'a',
    b: 'b',
};
```

Indexování typů

Indexování typů označuje možnost definovat typy, které lze indexovat předem neznámým klíčem, s využitím indexové signatury k určení typu vlastností, které nejsou explicitně deklarovány.

```ts
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Vrací a
```

Typ z hodnoty

Typ z hodnoty v TypeScriptu označuje automatické odvození typu z hodnoty nebo výrazu prostřednictvím odvozování typů.

```ts
const x = 'x'; // TypeScript odvodí 'x' jako řetězcový literál s 'const' (neměnný), ale rozšíří jej na 'string' s 'let' (lze znovu přiřadit).
```

Typ z návratové hodnoty funkce

Typ z návratové hodnoty funkce označuje možnost automaticky odvodit návratový typ funkce na základě její implementace. TypeScript tak může určit typ hodnoty vrácené funkcí bez explicitních typových anotací.

```
const add = (x: number, y: number) => x + y; // TypeScript dokáže odvodit, že návratový typ funkce je číslo
```

Typ z modulu

Typ z modulu označuje možnost používat exportované hodnoty modulu k automatickému odvození jejich typů. Když modul exportuje hodnotu s konkrétním typem, TypeScript může tuto informaci použít k automatickému odvození typu této hodnoty při jejím importu do jiného modulu.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r je typu number
```

Mapované typy

Mapované typy v TypeScriptu umožňují vytvářet nové typy na základě existujícího typu transformací každé vlastnosti pomocí mapovací funkce. Mapováním existujících typů můžete vytvářet nové typy, které představují stejné informace v jiném formátu. Mapovaný typ vytvoříte tak, že přistoupíte k vlastnostem existujícího typu pomocí operátoru `keyof` a poté je upravíte, čímž vznikne nový typ. V následujícím příkladu:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

definujeme MyMappedType tak, aby mapoval vlastnosti T a vytvořil nový typ, v němž je každá vlastnost polem svého původního typu. Pomocí toho vytváříme MyNewType, který představuje stejné informace jako MyType, ale každá vlastnost je polem.

Modifikátory mapovaných typů

Modifikátory mapovaných typů v TypeScriptu umožňují transformovat vlastnosti v existujícím typu:

- `readonly` nebo `+readonly`: Nastaví vlastnost v mapovaném typu jako pouze pro čtení.
- `-readonly`: Umožní měnit vlastnost v mapovaném typu.
- `?`: Označí vlastnost v mapovaném typu jako volitelnou.

Příklady:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // Všechny vlastnosti jsou označeny pouze pro čtení

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // Všechny vlastnosti jsou označeny jako měnitelné
```

```
type MyPartial<T> = { [P in keyof T]?: T[P] }; // Všechny vlastnosti jsou označeny jako volitelné
```

Podmíněné typy

Podmíněné typy představují způsob, jak vytvořit typ závislý na podmínce, kdy je vytvářený typ určen na základě výsledku podmínky. Definují se pomocí klíčového slova `extends` a ternárního operátoru pro podmíněný výběr mezi dvěma typy.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Typ true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Typ false
```

Distributivní podmíněné typy

Distributivní podmíněné typy jsou funkcí jazyka, která umožňuje distribuovat typ přes sjednocení typů tím, že se transformace aplikuje na každého člena sjednocení samostatně. To může být užitečné zejména při práci s mapovanými typy nebo typy vyšších řádů.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number | boolean | null
```

Odvozování typů pomocí infer v podmíněných typech

Klíčové slovo `infer` se v podmíněných typech používá k odvození (extrakci) typu generického parametru z typu, který na něm závisí. To umožňuje psát flexibilnější a opakovaně použitelné definice typů.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

Předdefinované podmíněné typy

Předdefinované podmíněné typy v TypeScriptu jsou vestavěné podmíněné typy poskytované jazykem. Jsou navrženy k provádění běžných transformací typů na základě vlastností daného typu.

`Exclude<UnionType, ExcludedType>`: Tento typ odstraní z Type všechny typy, které jsou přiřaditelné k ExcludedType.

`Extract<Type, Union>`: Tento typ extrahuje z Union všechny typy, které jsou přiřaditelné k Type.

`NonNullable<Type>`: Tento typ odstraní z Type null a undefined.

`ReturnType<Type>`: Tento typ extrahuje návratový typ funkce Type.

`Parameters<Type>`: Tento typ extrahuje typy parametrů funkce Type.

`Required<Type>`: Tento typ nastaví všechny vlastnosti v Type jako povinné.

`Partial<Type>`: Tento typ nastaví všechny vlastnosti v Type jako volitelné.

`Readonly<Type>`: Tento typ nastaví všechny vlastnosti v Type jako pouze pro čtení.

Šablonové sjednocené typy

Šablonové sjednocené typy lze použít ke spojování textu a manipulaci s ním v rámci typového systému, například:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active" | "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

Typ any

Typ `any` je speciální typ (univerzální nadtyp), který lze použít k reprezentaci jakéhokoli typu hodnoty (primitivních hodnot, objektů, polí, funkcí, chyb, symbolů). Často se používá v situacích, kdy typ hodnoty není znám v době kompilace, nebo při práci s hodnotami z externích API či knihoven, které nemají typové definice pro TypeScript.

Použitím typu `any` dáváte kompilátoru TypeScriptu najevo, že hodnoty mají být reprezentovány bez jakýchkoli omezení. Chcete-li v kódu maximalizovat typovou bezpečnost, zvažte následující:

- Omezte používání `any` na konkrétní případy, kdy je typ skutečně neznámý.

- Nevracejte z funkce hodnoty typu any, protože to oslabuje typovou bezpečnost kódu, který ji používá.
- Pokud potřebujete potlačit hlášení kompilátoru, použijte místo any direktivu `@ts-ignore`.

```
let value: any;
value = true; // Platné
value = 7; // Platné
```

Typ unknown

V TypeScriptu představuje typ unknown hodnotu neznámého typu. Na rozdíl od typu any, který umožňuje jakýkoli typ hodnoty, vyžaduje unknown před konkrétním použitím kontrolu typu nebo přetypování. S hodnotou typu unknown tedy nelze provádět žádné operace, aniž byste ji nejprve přetypovali nebo zúžili na konkrétnější typ.

Typ unknown lze přiřadit pouze typu any a samotnému typu unknown a představuje typově bezpečnou alternativu k any.

```
let value: unknown;

let value1: unknown = value; // Platné
let value2: any = value; // Platné
let value3: boolean = value; // Neplatné
let value4: number = value; // Neplatné

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

Typ void

Typ `void` se používá k označení toho, že funkce nevrací hodnotu.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

Typ never

Typ `never` představuje hodnoty, které nikdy nenastanou. Používá se k označení funkcí nebo výrazů, které se nikdy nevrátí nebo vyhodí chybu.

Například nekonečná smyčka:

```
const infiniteLoop = (): never => {
    while (true) {
        // provést nějakou akci
    }
};
```

Vyhození chyby:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Typ `never` je užitečný pro zajištění typové bezpečnosti a zachycení potenciálních chyb v kódu. V kombinaci s jinými typy a příkazy řízení toku pomáhá TypeScriptu analyzovat a odvozovat přesnější typy, například:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // posunout nahoru
```

```
            break;
        case 'down':
            // posunout dolů
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

Rozhraní a typ

Obvyklá syntaxe

V TypeScriptu definují rozhraní strukturu objektů a určují názvy a typy vlastností nebo metod, které objekt musí mít. Obvyklá syntaxe pro definici rozhraní v TypeScriptu je následující:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

Podobně pro definici typu:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` nebo `type TypeName`: Definuje název rozhraní. `property1: Type1`: Určuje vlastnosti rozhraní spolu s jejich odpovídajícími typy. Lze definovat více vlastností, každou oddělenou středníkem. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: Určuje metody rozhraní. Metody se definují svými názvy, za nimiž následuje seznam parametrů v závorkách a návratový typ. Lze definovat více metod, každou oddělenou středníkem.

Příklad rozhraní:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Příklad typu:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

V TypeScriptu se typy používají k definování struktury dat a vynucení typové kontroly. Pro definování typů v TypeScriptu existuje několik běžných způsobů zápisu podle konkrétního případu použití. Zde je několik příkladů:

Základní typy

```
let myNumber: number = 123; // typ number
let myBoolean: boolean = true; // typ boolean
let myArray: string[] = ['a', 'b']; // pole řetězců
let myTuple: [string, number] = ['a', 123]; // n-tice
```

Objekty a rozhraní

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

Sjednocení a průniky typů

```
type MyType = string | number; // Typ sjednocení
let myUnion: MyType = 'hello'; // Může být řetězec
myUnion = 123; // Nebo číslo

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Typ průniku
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Objekt s vlastnostmi name i age
```

Vestavěné primitivní typy

TypeScript má několik vestavěných primitivních typů, které lze použít k definování proměnných, parametrů funkcí a návratových typů:

- `number`: Představuje číselné hodnoty včetně celých čísel a čísel s plovoucí desetinnou čárkou.
- `string`: Představuje textová data.
- `boolean`: Představuje logické hodnoty, které mohou být buď pravda, nebo nepravda.
- `null`: Představuje nepřítomnost hodnoty.
- `undefined`: Představuje hodnotu, která nebyla přiřazena nebo definována.
- `symbol`: Představuje jedinečný identifikátor. Symboly se obvykle používají jako klíče vlastností objektů.
- `bigint`: Představuje celá čísla s libovolnou přesností.

- `any`: Představuje dynamický nebo neznámý typ. Proměnné typu any mohou obsahovat hodnoty libovolného typu a obcházejí typovou kontrolu.
- `void`: Představuje nepřítomnost jakéhokoli typu. Běžně se používá jako návratový typ funkcí, které nevracejí hodnotu.
- `never`: Představuje typ pro hodnoty, které nikdy nenastanou. Obvykle se používá jako návratový typ funkcí, které vyhodí chybu nebo vstoupí do nekonečné smyčky.

Běžné vestavěné objekty JS

TypeScript je nadmnožinou JavaScriptu; zahrnuje všechny běžně používané vestavěné objekty JavaScriptu. Rozsáhlý seznam těchto objektů najdete na webu s dokumentací Mozilla Developer Network (MDN): https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Zde je seznam některých běžně používaných vestavěných objektů JavaScriptu:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map

- Set
- Promise
- Intl

Přetížení

Přetížení funkcí v TypeScriptu umožňuje definovat více signatur pro jeden název funkce, takže ji lze volat různými způsoby. Zde je příklad:

```
// Přetížení
function sayHi(name: string): string;
function sayHi(names: string[]): string[];

// Implementace
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Platné
sayHi(['aa', 'bb']); // Platné
```

Zde je další příklad použití přetížení funkcí ve třídě (`class`):

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }
```

```
    // přetížení
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementace
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

Slučování a rozšiřování

Slučování a rozšiřování označují dva různé koncepty související s prací s typy a rozhraními.

Slučování umožňuje spojit více deklarací se stejným názvem do jediné definice, například když několikrát definujete rozhraní se stejným názvem:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
```

```
    b: 7,
};
```

Rozšiřování označuje možnost rozšířit existující typy či rozhraní nebo z nich dědit a vytvořit tak nové. Jde o mechanismus přidávání dalších vlastností nebo metod k existujícímu typu bez úpravy jeho původní definice. Příklad:

```
interface Animal {
    name: string;
    eat(): void;
}

interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

Rozdíly mezi typem a rozhraním

Slučování deklarací (rozšíření):

Rozhraní podporují slučování deklarací, což znamená, že můžete definovat více rozhraní se stejným názvem a TypeScript je sloučí do jediného rozhraní se spojenými vlastnostmi a metodami. Typy naproti tomu slučování deklarací nepodporují. To může být užitečné, když chcete přidat další funkčnost nebo

přizpůsobit existující typy bez úpravy původních definic nebo opravování chybějících či nesprávných typů.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Rozšiřování jiných typů/rozhraní:

Typy i rozhraní mohou rozšiřovat jiné typy/rozhraní, ale syntaxe se liší. U rozhraní používáte klíčové slovo `extends` k dědění vlastností a metod z jiných rozhraní. Rozhraní však nemůže rozšiřovat složitý typ, například sjednocení typů.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

U typů používáte operátor & ke spojení více typů do jediného typu (průniku).

```ts
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
    j: 'j',
};
```

Sjednocení a průniky typů:

Typy jsou při definování sjednocení a průniků typů flexibilnější. Pomocí klíčového slova `type` můžete snadno vytvářet sjednocení typů operátorem `|` a průniky typů operátorem `&`. Přestože rozhraní mohou také nepřímo reprezentovat sjednocení typů, nemají vestavěnou podporu pro průniky typů.

```ts
type Department = 'dep-x' | 'dep-y'; // Sjednocení

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Průnik
```

Příklad s rozhraními:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Sjednocení rozhraní
```

Třída

Obvyklá syntaxe třídy

Klíčové slovo `class` se v TypeScriptu používá k definování třídy. Níže vidíte příklad:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Klíčové slovo `class` se používá k definování třídy s názvem „Person“.

Třída má dvě soukromé vlastnosti: name typu `string` a age typu `number`.

Konstruktor je definován pomocí klíčového slova `constructor`. Přijímá name a age jako parametry a přiřazuje je odpovídajícím vlastnostem.

Třída má veřejnou metodu (`public`) s názvem sayHi, která vypisuje pozdrav.

K vytvoření instance třídy v TypeScriptu můžete použít klíčové slovo `new`, za nímž následuje název třídy a závorky `()`. Například:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Výstup: Hello, my name is John Doe and I am 25 years old.
```

Konstruktor

Konstruktory jsou speciální metody uvnitř třídy, které se používají k inicializaci vlastností objektu při vytváření instance třídy.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
```

```
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Konstruktor je možné přetížit pomocí následující syntaxe:

```
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

V TypeScriptu je možné definovat více přetížení konstruktoru, ale můžete mít pouze jednu implementaci, která musí být kompatibilní se všemi přetíženími; toho lze dosáhnout použitím volitelného parametru.

```
class Person {
    name: string;
    age: number;
```

```
    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

Soukromé a chráněné konstruktory

V TypeScriptu lze konstruktory označit jako soukromé nebo chráněné, čímž se omezuje jejich přístupnost a použití.

Soukromé konstruktory: Lze je volat pouze uvnitř samotné třídy. Soukromé konstruktory se často používají v situacích, kdy chcete vynutit návrhový vzor singleton nebo omezit vytváření instancí na tovární metodu uvnitř třídy.

Chráněné konstruktory: Chráněné konstruktory jsou užitečné, když chcete vytvořit základní třídu, jejíž instance by se neměly vytvářet přímo, ale kterou lze rozšiřovat podtřídami.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
        this.value = value;
    }
}

// Pokus o přímé vytvoření instance základní třídy způsobí chybu
// const baseObj = new BaseClass(); // Chyba: konstruktor třídy 'BaseClass' je chráněný.

// Vytvoření instance odvozené třídy
const derivedObj = new DerivedClass(10);
```

Modifikátory přístupu

Modifikátory přístupu `private`, `protected` a `public` slouží v TypeScriptu k řízení viditelnosti a přístupnosti členů tříd, jako jsou vlastnosti a metody. Tyto modifikátory jsou zásadní pro vynucení zapouzdření a stanovení hranic pro přístup k vnitřnímu stavu třídy a jeho úpravu.

Modifikátor `private` omezuje přístup ke členu třídy pouze na třídu, která jej obsahuje.

Modifikátor `protected` umožňuje přístup ke členu třídy uvnitř třídy, která jej obsahuje, a jejích odvozených tříd.

Modifikátor `public` poskytuje neomezený přístup ke členu třídy a umožňuje k němu přistupovat odkudkoli.

Get a Set

Gettery a settery jsou speciální metody, které umožňují definovat vlastní chování při přístupu k vlastnostem třídy a jejich úpravách. Umožňují zapouzdřit vnitřní stav objektu a přidat další logiku při získávání nebo nastavování hodnot vlastností. V TypeScriptu se gettery a settery definují pomocí klíčových slov `get` a `set`. Zde je příklad:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

Automatické přístupové metody ve třídách

TypeScript ve verzi 4.9 přidává podporu automatických přístupových metod, což je připravovaná funkce ECMAScriptu. Podobají se vlastnostem třídy, ale deklarují se klíčovým slovem „accessor“.

```
class Animal {
    accessor name: string;
```

```
    constructor(name: string) {
        this.name = name;
    }
}
```

Automatické přístupové metody se převádějí ze „syntaktického cukru“ na soukromé přístupové metody `get` a `set`, které pracují s nepřístupnou vlastností.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

this

V TypeScriptu odkazuje klíčové slovo `this` uvnitř metod nebo konstruktorů na aktuální instanci třídy. Umožňuje přistupovat k vlastnostem a metodám třídy a upravovat je v rámci jejího vlastního rozsahu platnosti. Poskytuje způsob, jak přistupovat k vnitřnímu stavu objektu a manipulovat s ním uvnitř jeho vlastních metod.

```
class Person {
    private name: string;
```

```
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}

const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

Vlastnosti deklarované v parametrech

Vlastnosti deklarované v parametrech umožňují deklarovat a inicializovat vlastnosti třídy přímo v parametrech konstruktoru, čímž se vyhnete opakujícímu se kódu. Například:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
        // Klíčová slova "private" a "public" v konstruktoru
          // automaticky deklarují a inicializují odpovídající
vlastnosti třídy.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

Abstraktní třídy

Abstraktní třídy se v TypeScriptu používají především pro dědičnost. Poskytují způsob, jak definovat společné vlastnosti a metody, které mohou podtřídy zdědit. To je užitečné, když chcete definovat společné chování a vynutit, aby podtřídy implementovaly určité metody. Umožňují vytvořit hierarchii tříd, v níž abstraktní základní třída poskytuje podtřídám sdílené rozhraní a společnou funkčnost.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Výstup: Whiskers meows.
```

S generiky

Třídy s generiky umožňují definovat znovupoužitelné třídy, které mohou pracovat s různými typy.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
```

```
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

Dekorátory

Dekorátory poskytují mechanismus pro přidávání metadat, úpravu chování, validaci nebo rozšiřování funkčnosti cílového prvku. Jsou to funkce, které se vykonávají za běhu. Na deklaraci lze použít více dekorátorů.

Dekorátory jsou experimentální funkce a následující příklady jsou kompatibilní pouze s TypeScriptem verze 5 nebo vyšší při použití ES6.

Ve verzích TypeScriptu před verzí 5 je třeba je povolit pomocí vlastnosti `experimentalDecorators` v souboru `tsconfig.json` nebo pomocí `--experimentalDecorators` na příkazovém řádku (následující příklad však nebude fungovat).

Mezi běžné případy použití dekorátorů patří:

- Sledování změn vlastností.
- Sledování volání metod.
- Přidávání dalších vlastností nebo metod.
- Validace za běhu.
- Automatická serializace a deserializace.
- Logování.
- Autorizace a autentizace.
- Ochrana před chybami.

Poznámka: Dekorátory pro verzi 5 neumožňují dekorovat parametry.

Typy dekorátorů:

Dekorátory tříd

Dekorátory tříd jsou užitečné pro rozšíření existující třídy, například přidáním vlastností či metod nebo shromažďováním instancí třídy. V následujícím příkladu přidáme metodu `toString`, která převádí třídu na řetězcovou reprezentaci.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}
```

```
@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Výpis:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

Dekorátor vlastnosti

Dekorátory vlastností jsou užitečné pro úpravu chování vlastnosti, například změnu inicializačních hodnot. V následujícím kódu máme skript, který nastavuje vlastnost tak, aby vždy obsahovala velká písmena:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
    @upperCase
```

```
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Výpis: HELLO!
```

Dekorátor metody

Dekorátory metod umožňují změnit nebo rozšířit chování metod. Níže je příklad jednoduchého loggeru:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

    function replacementMethod(this: This, ...args: Args): Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
}
```

```
new MyClass().sayHello();
```

Vypíše:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

Dekorátory getterů a setterů

Dekorátory getterů a setterů umožňují změnit nebo rozšířit chování přístupových metod třídy. Jsou užitečné například pro validaci přiřazování hodnot vlastnostem. Zde je jednoduchý příklad dekorátoru getteru:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}

class MyClass {
```

```
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Platné: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Vyhodí výjimku: Invalid!
```

Metadata dekorátorů

Metadata dekorátorů zjednodušují dekorátorům přidávání a využívání metadat v libovolné třídě. Mohou přistupovat k nové vlastnosti metadata na objektu kontextu, která může sloužit jako klíč pro primitivní hodnoty i objekty. K metadatům lze na třídě přistupovat prostřednictvím `Symbol.metadata`.

Metadata lze využít k různým účelům, například k ladění, serializaci nebo vkládání závislostí pomocí dekorátorů.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Jednoduchý polyfill

type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Kontext obsahuje metadata vlastnosti: DecoratorMetadata
```

```
function setMetadata(_target: any, context: Context) {
    // Nastavit primitivní hodnotu v objektu metadat
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Získání metadat

console.log(JSON.stringify(metadata));                    //
{"bar":true,"baz":true,"foo":true}
```

Dědičnost

Dědičnost označuje mechanismus, kterým může třída zdědit vlastnosti a metody z jiné třídy, označované jako základní třída nebo nadtřída. Odvozená třída, nazývaná také dceřiná třída nebo podtřída, může rozšířit a specializovat funkčnost základní třídy přidáním nových vlastností a metod nebo přepsáním těch stávajících.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }
```

```
    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Vytvoření instance základní třídy
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Vytvoření instance odvozené třídy
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript nepodporuje vícenásobnou dědičnost v tradičním smyslu a místo toho umožňuje dědit z jediné základní třídy. TypeScript podporuje více rozhraní. Rozhraní může definovat kontrakt pro strukturu objektu a třída může implementovat více rozhraní. To třídě umožňuje dědit chování a strukturu z více zdrojů.

```
interface Flyable {
    fly(): void;
}
```

```
interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Klíčové slovo `class` v TypeScriptu, podobně jako v JavaScriptu, bývá označováno jako syntaktický cukr. Bylo zavedeno v ECMAScriptu 2015 (ES6), aby nabídlo známější syntaxi pro vytváření objektů a práci s nimi způsobem založeným na třídách. Je však důležité zmínit, že TypeScript jako nadmnožina JavaScriptu se nakonec kompiluje do JavaScriptu, který zůstává ve své podstatě založen na prototypech.

Statické členy

TypeScript má statické členy. Ke statickým členům třídy můžete přistupovat pomocí názvu třídy následovaného tečkou, aniž byste museli vytvořit objekt.

```
class OfficeWorker {
    static memberCount: number = 0;
```

```
    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

Inicializace vlastností

V TypeScriptu existuje několik způsobů, jak inicializovat vlastnosti třídy:

Přímo v deklaraci:

V následujícím příkladu se tyto počáteční hodnoty použijí při vytvoření instance třídy.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

V konstruktoru:

```
class MyClass {
    property1: string;
    property2: number;

    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Pomocí parametrů konstruktoru:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
                // Hodnoty není nutné vlastnostem přiřazovat
explicitně.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

Přetěžování metod

Přetěžování metod umožňuje třídě mít více metod se stejným názvem, ale s různými typy parametrů nebo s různým počtem parametrů. To nám umožňuje volat metodu různými způsoby podle předaných argumentů.

```
class MyClass {
    add(a: number, b: number): number; // Signatura přetížení 1
    add(a: string, b: string): string; // Signatura přetížení 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
```

```
}

const r = new MyClass();
console.log(r.add(10, 5)); // Vypíše 15
```

Generika

Generika umožňují vytvářet znovupoužitelné komponenty a funkce, které mohou pracovat s více typy. Pomocí generik můžete parametrizovat typy, funkce a rozhraní, a umožnit jim tak pracovat s různými typy bez nutnosti je předem explicitně určit.

Generika umožňují vytvářet flexibilnější kód, který lze snáze znovu použít.

Generický typ

Při definování generického typu používáte lomené závorky (<>) k určení typových parametrů, například:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);

const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

Generické třídy

Generika lze použít také u tříd, které tak mohou pomocí typových parametrů pracovat s více typy. To je užitečné pro vytváření znovupoužitelných definic tříd, které mohou

pracovat s různými datovými typy při zachování typové bezpečnosti.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

Omezení generik

Generické parametry lze omezit pomocí klíčového slova `extends`, za nímž následuje typ nebo rozhraní, které musí typový parametr splňovat.

V následujícím příkladu musí mít `T` správně typovanou vlastnost `length`, aby byl platný:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
```

```
printLen({ length: 10 }); // 10
printLen(123); // Neplatné
```

Významnou funkcí generik zavedenou ve verzi 3.4 RC je odvozování typů funkcí vyššího řádu, které přenáší generické typové argumenty:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Tato funkčnost usnadňuje typově bezpečné programování ve stylu pointfree, které je běžné ve funkcionálním programování.

Kontextové zužování typů u generik

Kontextové zužování typů u generik je mechanismus v TypeScriptu, který kompilátoru umožňuje zúžit typ generického parametru na základě kontextu, ve kterém se používá. Je užitečné při práci s generickými typy v podmíněných příkazech:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Hodnota je zúžena na typ 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Hodnota je zúžena na typ 'number'
        console.log(value.toFixed(2));
```

```
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

Vymazané strukturální typy

V TypeScriptu nemusejí objekty odpovídat konkrétnímu, přesně určenému typu. Pokud například vytvoříme objekt, který splňuje požadavky rozhraní, můžeme jej použít na místech, kde je toto rozhraní vyžadováno, i když mezi nimi neexistuje žádná explicitní vazba. Příklad:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
};

log(obj); // Platné
```

Jmenné prostory

V TypeScriptu se jmenné prostory používají k uspořádání kódu do logických celků, zabraňují kolizím názvů a poskytují způsob, jak seskupit související kód. Použití klíčového slova `export` umožňuje přístup ke jmennému prostoru z vnějších modulů.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

Symboly

Symboly jsou primitivním datovým typem, který představuje neměnnou hodnotu se zaručenou globální jedinečností po celou dobu běhu programu.

Symboly lze použít jako klíče vlastností objektů a poskytují způsob, jak vytvářet neenumerovatelné vlastnosti.

```
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Ve WeakMaps a WeakSets lze nyní jako klíče používat symboly.

Direktivy s trojitým lomítkem

Direktivy s trojitým lomítkem jsou speciální komentáře, které kompilátoru poskytují pokyny ke zpracování souboru. Tyto direktivy začínají třemi po sobě jdoucími lomítky (`///`), obvykle se umísťují na začátek souboru TypeScriptu a nemají žádný vliv na chování za běhu.

Direktivy s trojitým lomítkem se používají k odkazování na externí závislosti, určení chování při načítání modulů, povolování nebo zakazování určitých funkcí kompilátoru a dalším účelům. Několik příkladů:

Odkaz na deklarační soubor:

```
/// <reference path="path/to/declaration/file.d.ts" />
```

Určení formátu modulu:

```
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Povolení voleb kompilátoru, v následujícím příkladu striktního režimu:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

Manipulace s typy

Vytváření typů z typů

Nové typy lze vytvářet skládáním, úpravou nebo transformací existujících typů.

Průniky typů (`&`):

Umožňují spojit více typů do jednoho typu:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Průnik A a B
const obj: C = { foo: 42, bar: 'hello' };
```

Sjednocení typů (|):

Umožňují definovat typ, který může být jedním z několika typů:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Mapované typy:

Umožňují transformovat vlastnosti existujícího typu a vytvořit tak nový typ:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Vlastnosti se stanou
dostupnými pouze pro čtení
```

Podmíněné typy:

Umožňují vytvářet typy na základě určitých podmínek:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

Typy s indexovaným přístupem

V TypeScriptu lze pomocí indexu `Type[Key]` přistupovat k typům vlastností uvnitř jiného typu a upravovat je.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

Pomocné typy

K manipulaci s typy lze použít několik vestavěných pomocných typů. Níže je uveden seznam nejčastěji používaných:

Awaited<T>

Vytváří typ, který rekurzivně rozbaluje typy Promise.

```
type A = Awaited<Promise<string>>; // string
```

Partial<T>

Vytváří typ, v němž jsou všechny vlastnosti T nastaveny jako volitelné.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

Required<T>

Vytváří typ, v němž jsou všechny vlastnosti T nastaveny jako povinné.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

Readonly<T>

Vytváří typ, v němž jsou všechny vlastnosti T nastaveny pouze pro čtení.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Neplatné
```

Record<K, T>

Vytváří typ se sadou vlastností K typu T.

```
type Product = {
    name: string;
    price: number;
};
```

```ts
const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

Pick<T, K>

Vytváří typ výběrem zadaných vlastností K z T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

Omit<T, K>

Vytváří typ vynecháním zadaných vlastností K z T.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

Exclude<T, U>

Vytváří typ vyloučením všech hodnot typu U z T.

```ts
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

Extract<T, U>

Vytváří typ extrakcí všech hodnot typu U z T.

```
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

NonNullable<T>

Vytváří typ vyloučením null a undefined z T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

Parameters<T>

Extrahuje typy parametrů z typu funkce T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

ConstructorParameters<T>

Extrahuje typy parametrů z typu konstruktorové funkce T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type PersonConstructorParams = ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

ReturnType<T>

Extrahuje návratový typ z typu funkce T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

InstanceType<T>

Extrahuje typ instance z typu třídy T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

ThisParameterType<T>

Extrahuje typ parametru ‘this’ z typu funkce T.

```
interface Person {
    name: string;
    greet(this: Person): void;
```

```
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

OmitThisParameter<T>

Odstraňuje parametr ‘this’ z typu funkce T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      + this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; // () => string
```

ThisType<T>

Slouží jako značka pro kontextový typ `this`.

```
type Logger = {
    log: (error: string) => void;
};

let helperFunctions: { [name: string]: Function } & ThisType<Logger> = {
    hello: function () {
          this.log('some error'); // Platné, protože "log" je součástí "this".
        this.update(); // Neplatné
    },
};
```

Uppercase<T>

Převádí název vstupního typu T na velká písmena.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

Lowercase<T>

Převádí název vstupního typu T na malá písmena.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

Capitalize<T>

Převádí první písmeno názvu vstupního typu T na velké písmeno.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

Uncapitalize<T>

Převádí první písmeno názvu vstupního typu T na malé písmeno.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

NoInfer<T>

NoInfer je pomocný typ navržený k blokování automatického odvozování typů v rámci generické funkce.

Příklad:

```
// Automatické odvozování typů v rámci generické funkce.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Typ je zde ("a" | "b" | "c")
[]
```

S NoInfer:

```
// Příklad funkce, která používá NoInfer k zabránění odvozování typů
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Chyba: argument typu '"c"' nelze přiřadit parametru typu '"a" | "b"'.
```

Ostatní

Chyby a zpracování výjimek

TypeScript umožňuje zachytávat a zpracovávat chyby pomocí standardních mechanismů JavaScriptu pro zpracování chyb:

Bloky Try-Catch-Finally:

```
try {
    // Kód, který může vyhodit chybu
} catch (error) {
    // Zpracování chyby
} finally {
    // Kód, který se provede vždy, finally je volitelné
}
```

Můžete také zpracovávat různé typy chyb:

```
try {
    // Kód, který může vyhodit různé typy chyb
} catch (error) {
    if (error instanceof TypeError) {
        // Zpracování TypeError
    } else if (error instanceof RangeError) {
        // Zpracování RangeError
```

```
    } else {
        // Zpracování ostatních chyb
    }
}
```

Vlastní typy chyb:

Konkrétnější chyby lze definovat rozšířením třídy (`class`) Error:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

Mixinové třídy

Mixinové třídy umožňují kombinovat a skládat chování více tříd do jediné třídy. Poskytují způsob, jak opakovaně používat a rozšiřovat funkcionalitu bez nutnosti hlubokých řetězců dědičnosti.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
```

```
    deselect() {
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Rozšířit MyClass o chování Identifiable a Selectable
interface MyClass extends Identifiable, Selectable {}

// Funkce pro aplikaci mixinů na třídu
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Aplikovat mixiny na MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

Asynchronní jazykové funkce

Protože je TypeScript nadmnožinou JavaScriptu, má vestavěné asynchronní jazykové funkce JavaScriptu, jako jsou:

Promise:

Promise představují způsob, jak zpracovávat asynchronní operace a jejich výsledky pomocí metod jako `.then()` a `.catch()` pro zpracování úspěšných a chybových stavů.

Další informace: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Klíčová slova async/await poskytují syntaxi pro práci s Promise, která více připomíná synchronní kód. Klíčové slovo `async` slouží k definici asynchronní funkce a klíčové slovo `await` se používá uvnitř asynchronní funkce k pozastavení vykonávání, dokud není Promise splněna nebo odmítnuta.

Další informace: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Následující API jsou v TypeScriptu dobře podporována:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Iterátory a generátory

Iterátory i generátory jsou v TypeScriptu dobře podporovány.

Iterátory jsou objekty, které implementují protokol iterátoru a poskytují způsob, jak postupně přistupovat k jednotlivým prvkům kolekce nebo posloupnosti. Jde o strukturu, která obsahuje ukazatel na další prvek v iteraci. Mají metodu `next()`, která vrací další hodnotu v posloupnosti spolu s booleovskou hodnotou udávající, zda je posloupnost dokončena (`done`).

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
    }
```

```
    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generátory jsou speciální funkce definované pomocí syntaxe `function*`, která zjednodušuje vytváření iterátorů. Pomocí klíčového slova `yield` definují posloupnost hodnot a při požadavcích na hodnoty automaticky pozastavují a obnovují vykonávání.

Generátory usnadňují vytváření iterátorů a jsou zvláště užitečné při práci s velkými nebo nekonečnými posloupnostmi.

Příklad:

```
function* numberGenerator(start: number, end: number):
Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript také podporuje asynchronní iterátory a asynchronní generátory.

Další informace:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

Referenční příručka TsDocs JSDoc

Při práci s kódovou základnou v JavaScriptu lze TypeScriptu pomoci odvodit správný typ pomocí komentářů JSDoc s dodatečnými anotacemi poskytujícími informace o typech.

Příklad:

```
/**
 * Vypočítá mocninu zadaného čísla
 * @constructor
 * @param {number} base - Základ výrazu
 * @param {number} exponent - Exponent výrazu
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Úplná dokumentace je k dispozici na tomto odkazu: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Od verze 3.7 lze generovat definice typů .d.ts ze syntaxe JSDoc v JavaScriptu. Další informace naleznete zde: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

@types

Balíčky v organizaci @types využívají speciální konvenci pojmenování balíčků, které poskytují definice typů pro existující knihovny nebo moduly JavaScriptu. Například použitím:

```
npm install --save-dev @types/lodash
```

nainstalujete definice typů pro `lodash` do svého aktuálního projektu.

Chcete-li přispět k definicím typů balíčku `@types`, odešlete prosím pull request do https://github.com/DefinitelyTyped/DefinitelyTyped.

JSX

JSX (JavaScript XML) je rozšíření syntaxe jazyka JavaScript, které umožňuje psát kód podobný HTML v souborech JavaScriptu nebo TypeScriptu. Běžně se používá v Reactu k definování struktury HTML.

TypeScript rozšiřuje možnosti JSX tím, že poskytuje kontrolu typů a statickou analýzu.

Chcete-li používat JSX, musíte nastavit volbu kompilátoru `jsx` v souboru `tsconfig.json`. Dvě běžné možnosti konfigurace:

- “preserve”: generuje soubory .jsx s nezměněným JSX. Tato volba říká TypeScriptu, aby zachoval syntaxi JSX tak, jak je, a během kompilace ji netransformoval. Tuto volbu můžete použít, pokud máte samostatný nástroj, například Babel, který transformaci provádí.
- “react”: zapíná vestavěnou transformaci JSX v TypeScriptu. Bude použito React.createElement.

Všechny možnosti jsou k dispozici zde: https://www.typescriptlang.org/tsconfig#jsx

Moduly ES6

TypeScript podporuje ES6 (ECMAScript 2015) a mnoho následujících verzí. To znamená, že můžete používat syntaxi ES6, například šipkové funkce, šablonové literály, třídy, moduly, destrukturování a další.

Chcete-li ve svém projektu povolit funkce ES6, můžete v tsconfig.json zadat vlastnost `target`.

Příklad konfigurace:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Operátor umocňování ES7

Operátor umocňování (**) vypočítá hodnotu získanou umocněním prvního operandu na mocninu danou druhým operandem. Funguje podobně jako `Math.pow()`, ale navíc může jako operandy přijímat hodnoty BigInt. TypeScript tento operátor plně podporuje při nastavení `target` v souboru tsconfig.json na `es2016` nebo vyšší.

```
console.log(2 ** (2 ** 2)); // 16
```

Příkaz for-await-of

Jde o funkci JavaScriptu plně podporovanou v TypeScriptu, která umožňuje iterovat přes asynchronní iterovatelné objekty s cílovou verzí `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

Metavlastnost new.target

V TypeScriptu můžete použít metavlastnost `new.target`, která umožňuje zjistit, zda došlo k vyvolání funkce nebo konstruktoru pomocí operátoru new. Umožňuje rozpoznat, zda byl objekt vytvořen v důsledku volání konstruktoru.

```
class Parent {
    constructor() {
         console.log(new.target); // Vypíše funkci konstruktoru
použitou k vytvoření instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

Výrazy dynamického importu

Pomocí návrhu ECMAScriptu pro dynamický import, který je v TypeScriptu podporován, lze moduly načítat podmíněně nebo odloženě na vyžádání.

Syntaxe výrazů dynamického importu v TypeScriptu je následující:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
         const widget = await import('./widget'); // Dynamický
import
        widget.render(container);
    }
}

renderWidget();
```

“tsc –watch”

Tento příkaz spustí kompilátor TypeScriptu s parametrem `--watch`, který umožňuje automaticky znovu kompilovat soubory TypeScriptu při každé jejich změně.

```
tsc --watch
```

Od verze TypeScriptu 4.9 se sledování souborů spoléhá především na události souborového systému a automaticky přechází na pravidelné dotazování, pokud nelze zřídit sledování založené na událostech.

Operátor potvrzení nenulové hodnoty

Operátor potvrzení nenulové hodnoty (postfix !), označovaný také jako potvrzení zaručeného přiřazení, je funkce TypeScriptu, která umožňuje deklarovat, že proměnná nebo vlastnost není null ani undefined, i když statická analýza typů TypeScriptu naznačuje, že by mohla být. Díky této funkci lze odstranit veškeré explicitní kontroly.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

Deklarace s výchozími hodnotami

Deklarace s výchozími hodnotami se používají, když je proměnné nebo parametru přiřazena výchozí hodnota. To znamená, že pokud pro danou proměnnou nebo parametr není poskytnuta žádná hodnota, použije se místo ní výchozí hodnota.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

Volitelné řetězení

Operátor volitelného řetězení `?.` funguje jako běžný tečkový operátor (`.`) pro přístup k vlastnostem nebo metodám. Hodnoty null a undefined však ošetřuje ukončením výrazu a vrácením `undefined` namísto vyvolání chyby.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

Operátor slučování nulových hodnot

Operátor slučování nulových hodnot `??` vrací hodnotu na pravé straně, pokud je hodnota na levé straně `null` nebo `undefined`; jinak vrací hodnotu na levé straně.

```
const foo = null ?? 'foo';
console.log(foo); // foo
```

```
const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

Typy šablonových literálů

Typy šablonových literálů umožňují manipulovat s řetězcovými hodnotami na úrovni typů a generovat nové řetězcové typy na základě existujících. Jsou užitečné pro vytváření výstižnějších a přesnějších typů z operací s řetězci.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

Přetěžování funkcí

Přetěžování funkcí umožňuje definovat více signatur pro stejný název funkce, každou s jinými typy parametrů a návratovými typy. Při volání přetížené funkce TypeScript použije předané argumenty k určení správné signatury funkce:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}
```

```
makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

Rekurzivní typy

Rekurzivní typ je typ, který může odkazovat sám na sebe. To je užitečné pro definování datových struktur s hierarchickou nebo rekurzivní strukturou (potenciálně nekonečným vnořením), například spojových seznamů, stromů a grafů.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

Rekurzivní podmíněné typy

V TypeScriptu lze pomocí logiky a rekurze definovat složité vztahy mezi typy. Rozeberme si to jednoduše:

Podmíněné typy umožňují definovat typy na základě booleovských podmínek:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Rekurze znamená definici typu, která v rámci své vlastní definice odkazuje sama na sebe:

```
type Json = string | number | boolean | null | Json[] | { [key: string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
```

```
    prop3: {
        prop4: [],
    },
};
```

Rekurzivní podmíněné typy kombinují podmíněnou logiku a rekurzi. To znamená, že definice typu může prostřednictvím podmíněné logiky záviset sama na sobě a vytvářet tak složité a flexibilní vztahy mezi typy.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 |
1 | 6
```

Podpora modulů ECMAScript v Node

Node.js přidal podporu modulů ECMAScript od verze 15.3.0 a TypeScript podporuje moduly ECMAScript pro Node.js od verze 4.7. Tuto podporu lze zapnout pomocí vlastnosti `module` s hodnotou `nodenext` v souboru tsconfig.json. Zde je příklad:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
  }
}
```

Node.js podporuje dvě přípony souborů pro moduly: `.mjs` pro moduly ES a `.cjs` pro moduly CommonJS. Odpovídající přípony souborů v TypeScriptu jsou `.mts` pro moduly ES a `.cts` pro moduly CommonJS. Když kompilátor TypeScriptu transpiluje tyto soubory do JavaScriptu, vytvoří soubory `.mjs` a `.cjs`.

Chcete-li ve svém projektu používat moduly ES, můžete v souboru package.json nastavit vlastnost `type` na "module". Tím sdělíte Node.js, aby projekt považoval za projekt s moduly ES.

TypeScript navíc podporuje deklarace typů v souborech .d.ts. Tyto deklarační soubory poskytují informace o typech pro knihovny nebo moduly napsané v TypeScriptu a umožňují ostatním vývojářům využívat je s kontrolou typů a automatickým doplňováním TypeScriptu.

Aserční funkce

V TypeScriptu jsou aserční funkce takové funkce, které na základě své návratové hodnoty indikují ověření konkrétní podmínky. V nejjednodušší podobě aserční funkce zkoumá předaný predikát a vyvolá chybu, když se predikát vyhodnotí jako false.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Nebo ji lze deklarovat jako výraz funkce:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Aserční funkce mají podobné vlastnosti jako typoví strážci. Typoví strážci byli původně zavedeni k provádění kontrol za běhu a k zajištění typu hodnoty v určitém rozsahu platnosti. Konkrétně je typový strážce funkce, která vyhodnocuje typový predikát a vrací booleovskou hodnotu udávající, zda je predikát pravdivý, či nepravdivý. To se mírně liší od aserčních funkcí, jejichž záměrem je při nesplnění predikátu vyvolat chybu místo vrácení false.

Příklad typového strážce:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

Variadické typy n-tic

Variadické typy n-tic jsou funkcí zavedenou v TypeScriptu verze 4.0, proto si nejprve připomeňme, co je n-tice:

Typ n-tice je pole, které má definovanou délku a u něhož je znám typ každého prvku:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Termín „variadický“ znamená neurčitou aritu (přijímání proměnného počtu argumentů).

Variadická n-tice je typ n-tice, který má všechny výše uvedené vlastnosti, ale jeho přesná podoba ještě není definována:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
```

```
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

V předchozím kódu vidíme, že podobu n-tice určuje předaný generický parametr `T`.

Variadické n-tice mohou přijímat více generických parametrů, díky čemuž jsou velmi flexibilní:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

S novými variadickými n-ticemi můžeme využít následující:

- Rozvinutí v syntaxi typů n-tic mohou být nyní generická, takže můžeme reprezentovat operace vyššího řádu nad n-ticemi a poli, i když neznáme skutečné typy, nad kterými pracujeme.
- Zbytkové prvky se mohou v n-tici vyskytovat kdekoli.

Příklad:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

Obalové typy

Obalové typy označují obalové objekty, které slouží k reprezentaci primitivních typů jako objektů. Tyto obalové objekty poskytují další funkcionalitu a metody, které nejsou dostupné přímo u primitivních hodnot.

Když přistupujete k metodě jako `charAt` nebo `normalize` u primitivní hodnoty typu `string`, JavaScript ji zabalí do objektu `String`, zavolá metodu a poté objekt zahodí.

Ukázka:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript toto rozlišení reprezentuje tím, že poskytuje samostatné typy pro primitivní hodnoty a jejich odpovídající objektové obaly:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Obalové typy obvykle nejsou potřeba. Vyhněte se používání obalových typů a místo nich používejte primitivní typy, například `string` namísto `String`.

Kovariance a kontravariance v TypeScriptu

Kovariance a kontravariance popisují vztahy mezi typy uvnitř generických typů. `Dog` je `Animal`, ale ne každý `Animal` je `Dog`.

Kovariance (vytváření hodnot): funkce vracející `Dog` může zastoupit funkci vracející `Animal`. Každý pes je zvíře, ale funkce vracející libovolné zvíře může vrátit kočku.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;

produceAnimal = produceDog; // Platné
produceDog = produceAnimal; // Chyba
```

Kontravariance (přijímání hodnot): funkce přijímající libovolné `Animal` může zastoupit funkci přijímající `Dog`. Zvládne i psy; funkce určená jen pro psy nemůže bezpečně přijmout každé zvíře.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Platné
consumeAnimal = consumeDog; // Chyba při zapnutém strictFunctionTypes
```

Šipky ukazují směr bezpečného přiřazení:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

Měnitelná pole: TypeScript dovoluje také přiřadit `Dog[]` do `Animal[]`. Obě proměnné odkazují na stejné pole; přes obecnější typ lze vložit `Animal`, které není pes.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Povoleno, ale nebezpečné
// dogs nyní obsahuje Animal, který není Dog.
```

Nastavení překladače: při zapnutém `strictFunctionTypes` se parametry samostatných funkcí kontrolují kontravariantně. Bez této volby může TypeScript povolit oba směry (bivarianci), i když nejsou bezpečné. Parametry deklarované v metodách a konstruktorech tvoří výjimku.

Volitelné anotace variance pro typové parametry

Od TypeScriptu 4.7 označuje `out` kovarianci, `in` kontravarianci a `in out` invarianci. TypeScript varianci obvykle odvodí. Anotace musí odpovídat použití generického parametru a nemění jeho chování libovolně.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

Indexové signatury se vzory šablonových řetězců

Indexové signatury se vzory šablonových řetězců umožňují definovat flexibilní indexové signatury pomocí vzorů šablonových řetězců. Tato funkce umožňuje vytvářet objekty, které lze indexovat pomocí konkrétních vzorů řetězcových klíčů, a poskytuje tak větší kontrolu a přesnost při přístupu k vlastnostem a manipulaci s nimi.

TypeScript od verze 4.4 umožňuje indexové signatury pro symboly a vzory šablonových řetězců.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Klíč unikátního symbolu
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

Operátor satisfies

Operátor `satisfies` umožňuje zkontrolovat, zda daný typ splňuje konkrétní rozhraní nebo podmínku. Jinými slovy zajišťuje, že typ má všechny požadované vlastnosti a metody

určitého rozhraní. Je to způsob, jak zajistit, že proměnná odpovídá definici typu. Zde je příklad:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Typová anotace pomocí `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// Na následujících řádcích TypeScript nedokáže správně odvodit typy
user.attributes?.map(console.log);    // Vlastnost 'map' neexistuje v typu 'string | string[]'. Vlastnost 'map' neexistuje v typu 'string'.
user.nickName; // string | string[] | undefined

// Typové tvrzení pomocí `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Ani zde TypeScript nedokáže správně odvodit typy
user2.attributes?.map(console.log);    // Vlastnost 'map' neexistuje v typu 'string | string[]'. Vlastnost 'map' neexistuje v typu 'string'.
user2.nickName; // string | string[] | undefined

// Pomocí operátoru `satisfies` nyní můžeme správně odvodit typy
const user3 = {
    name: 'Simon',
    nickName: undefined,
```

```
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log);   // TypeScript správně
odvodí: string[]
user3.nickName; // TypeScript správně odvodí: undefined
```

Importy a exporty pouze typů

Importy a exporty pouze typů umožňují importovat nebo exportovat typy bez importování nebo exportování hodnot či funkcí spojených s těmito typy. To může být užitečné pro zmenšení velikosti výsledného balíčku.

Pro importy pouze typů můžete použít klíčové slovo `import type`.

TypeScript umožňuje v importech pouze typů používat přípony deklaračních i implementačních souborů (.ts, .mts, .cts a .tsx) bez ohledu na nastavení `allowImportingTsExtensions`.

Například:

```
import type { House } from './house.ts';
```

Podporovány jsou následující formy:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

Deklarace using a explicitní správa prostředků

Deklarace `using` představuje neměnnou vazbu s rozsahem platnosti v bloku, podobnou `const`, která slouží ke správě uvolnitelných prostředků. Při inicializaci hodnotou se zaznamená metoda `Symbol.dispose` této hodnoty a následně se vykoná při opuštění bloku, který deklaraci obsahuje.

Vychází to z funkce správy prostředků v ECMAScriptu, která je užitečná pro provádění nezbytných úklidových operací po vytvoření objektu, například uzavírání spojení, mazání souborů a uvolňování paměti.

Poznámky:

- Vzhledem k nedávnému zavedení v TypeScriptu verze 5.2 většina běhových prostředí postrádá nativní podporu. Budete potřebovat polyfilly pro: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Dále budete muset nakonfigurovat svůj tsconfig.json následovně:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Příklad:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Jednoduchý polyfill

const doWork = (): Disposable => {
```

```
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Zdroj je deklarován
    console.log(2);
} // Zdroj je uvolněn (např. se vyhodnotí `work[Symbol.dispose]
()`)

console.log(3);
```

Kód vypíše:

```
1
2
disposed
3
```

Prostředek, který lze uvolnit, musí splňovat rozhraní `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Deklarace `using` zaznamenávají operace uvolnění prostředků do zásobníku, čímž zajišťují jejich uvolnění v opačném pořadí, než v jakém byly deklarovány:

```
{
    using j = getA(),
        y = getB();
```

```
    using k = getC();
} // uvolní `C`, poté `B`, poté `A`.
```

Je zaručeno, že prostředky budou uvolněny, i když následuje další kód nebo dojde k výjimkám. Při uvolňování tak může dojít k vyvolání výjimky, která případně potlačí jinou výjimku. Pro zachování informací o potlačených chybách je zavedena nová nativní výjimka `SuppressedError`.

Deklarace await using

Deklarace `await using` spravuje asynchronně uvolnitelný prostředek. Hodnota musí mít metodu `Symbol.asyncDispose`, na jejíž dokončení se počká na konci bloku.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Zdroj je deklarován
}   //   Zdroj   je   uvolněn   (např.   se   vyhodnotí   `await
work[Symbol.asyncDispose]()`)
```

Asynchronně uvolnitelný prostředek musí splňovat buď rozhraní `Disposable`, nebo `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Jednoduchý polyfill

class DatabaseConnection implements AsyncDisposable {
    // Metoda volaná při asynchronním uvolnění objektu
    [Symbol.asyncDispose]() {
        // Zavřít spojení a vrátit promise
```

```
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
            await new Promise(resolve => setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
      // Vytvořit nové spojení a asynchronně jej uvolnit při
opuštění rozsahu platnosti
       await using connection = new DatabaseConnection(); //
Zdroj je deklarován
    console.log('Doing some work...');
}   //   Zdroj   je   uvolněn   (např.   se   vyhodnotí   `await
connection[Symbol.asyncDispose]()`)

doWork();
```

Kód vypíše:

```
Doing some work...
Closing the connection...
Connection closed.
```

Deklarace `using` a `await using` jsou povoleny v příkazech: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

Atributy importu

Atributy importu (štítky pro importy) v TypeScriptu 5.3 říkají běhovému prostředí, jak zacházet s moduly (JSON atd.). Zvyšují bezpečnost tím, že zajišťují jednoznačné importy, a jsou v souladu s Content Security Policy (CSP) pro bezpečnější načítání

prostředků. TypeScript zajišťuje jejich platnost, ale jejich interpretaci pro konkrétní zpracování modulů ponechává běhovému prostředí.

Příklad:

```
import config from './config.json' with { type: 'json' };
```

s dynamickým importem:

```
const config = import('./config.json', { with: { type: 'json' }
});
```

Kontrola syntaxe regulárních výrazů

Od verze 5.5.4 TypeScript při kompilaci kontroluje literály regulárních výrazů na běžné chyby (např. neplatnou syntaxi, chybné zpětné odkazy nebo funkce nepodporované cílovou verzí JS). Pomáhá odhalit chyby dříve, ale nekontroluje řetězce new RegExp(“…”).

```
let r = /(a)\2/; // Chyba: tento zpětný odkaz odkazuje na
neexistující skupinu.
```

import defer

`import defer` umožňuje načíst modul, ale odložit jeho vykonání, dokud z něj něco skutečně nepoužijete. Pomáhá to předejít zbytečné práci a vedlejším účinkům.

- Funguje pouze s: `import defer * as name from "module"`
- Kód se spustí pouze při přístupu k exportu